



VIT-AP UNIVERSITY

CINEMATCH

MOVIE RECOMMENDATION AND ANALYTICS SYSTEM

A NoSQL Database Case Study Using MongoDB

Query, Aggregation and Recommendation Analysis of the MovieLens Dataset

Registration Number	Name
24MIC7198	V. RUTHVIK
24MIC7181	K. TEJASH NARAYANA
24MIC7203	B. HARSHITH

Under The Guidance Of : Dr. VASAVI SANIKOMMU

School of Computer Science and Engineering (SCOPE)

VIT-AP University, Amaravati, Andhra Pradesh – 522241

Course: NoSQL Databases

Abstract

This case study presents **CineMatch**, a movie recommendation and analytics system built on MongoDB. The publicly available MovieLens *ml-latest-small* dataset – 100,836 ratings and 3,683 tags applied to 9,742 movies by 610 users – was imported into a MongoDB database named *cinematch* as four collections (movies, ratings, tags, links). The raw CSV data was then cleaned and enriched inside MongoDB itself using pipeline-style updates: titles with a trailing article were normalised, the release year was extracted from the title with `$regexFind`, the pipe-delimited genre string was split into an array with `$split`, and Unix timestamps were converted to `ISODate` values.

A catalogue of 58 queries was executed in `mongosh`, progressing from basic CRUD, filtering and projection to advanced aggregation pipelines that use `$lookup` (joins), `$unwind`, `$group`, `$facet`, `$bucket`, `$setIntersection`, `$stdDevPop` and `$merge`. The analysis covers movie and genre statistics, rating distributions, user behaviour over time, tag analysis, and three recommendation techniques – content-based (genre overlap), collaborative ("users who liked X also liked"), and a Bayesian weighted rating that balances quality with popularity. Indexes (single-field, unique, multikey and text) were created and their effect verified with `explain()`. Finally, a web dashboard (Node.js + Express + React) exposes the same pipelines through a REST API so that the results can be explored interactively.

1. Introduction

Online streaming platforms and movie databases generate enormous amounts of user-interaction data: every rating, every tag and every search is an event that can be analysed to understand audience taste and to recommend what a user should watch next. Such data is semi-structured, grows continuously, and is accessed mostly through analytical questions ("what are the highest-rated crime films of the 1990s?", "what do fans of The Matrix also enjoy?") rather than through rigid transactional updates.

Document-oriented NoSQL databases such as MongoDB are well suited to this workload. Documents map naturally to the JSON records produced by applications, related data can be either embedded or referenced, and the aggregation framework allows complex multi-stage analysis to be performed inside the database without exporting the data to another tool. This case study uses MongoDB to store the MovieLens dataset and answers a wide range of analytical and recommendation questions using only queries and aggregation pipelines.

2. Problem Statement

Movie rating data is spread across several related files (movies, ratings, tags, links) that use different structures: a movie has a title and a multi-valued genre field, a rating links a user to a movie with a score and a timestamp, and a tag is free text. Viewers and platform operators want answers to questions that require combining these files – the average rating of each genre, the most controversial movies, the most active users, and, most importantly, which unseen movies a particular user is likely to enjoy.

Answering such questions with a relational schema requires many joins and normalisation of the multi-valued genre field. The challenge addressed in this case study is to model the MovieLens data in a NoSQL document database, clean and enrich it in place, and use MongoDB's query and aggregation capabilities to produce descriptive analytics and recommendation results efficiently, verifying performance with indexes and execution plans.

3. Aim

The aim of this project is to design a MongoDB database for movie rating data and to demonstrate, through a comprehensive set of queries and aggregation pipelines, how a document-oriented NoSQL database can be used to analyse user ratings, genres, tags and viewing behaviour, and to generate movie recommendations for users – supported by a web dashboard that presents the results interactively.

4. Objectives

1. To import the MovieLens CSV dataset into MongoDB as separate collections and verify the import.
2. To clean and enrich the data inside MongoDB using pipeline-style updateMany operations (\$regexFind, \$concat, \$split, \$toDate).
3. To perform CRUD operations and basic queries using filtering, projection, sorting, regular expressions and text search.
4. To analyse rating behaviour: distribution of star values, overall statistics, most-rated, highest-rated, lowest-rated and most controversial movies.
5. To analyse genres by joining ratings with movies (\$lookup) and unwinding the genre array (\$unwind).
6. To analyse users: most active users, harsh vs. generous critics (\$facet), activity bands (\$bucket), and rating activity over time.
7. To analyse the free-text tags applied by users.
8. To implement three recommendation approaches in pure MongoDB: content-based genre similarity, collaborative filtering and Bayesian weighted rating.
9. To create appropriate indexes (single, unique, multikey, text) and verify their effect with explain().
10. To materialise pre-computed statistics with \$merge and expose the analysis through a REST API and an interactive web dashboard.

5. Dataset Description

The project uses the **MovieLens ml-latest-small** dataset published by GroupLens Research (University of Minnesota) and mirrored on Kaggle as "*Popular Movies Datasets – 9742 Movies*". It describes 5-star rating and free-text tagging activity from the MovieLens recommendation service between 29 March 1996 and 24 September 2018. Users were selected at random and every user has rated at least 20 movies; no demographic information is included.

File	Rows	Columns	Description
movies.csv	9,742	movieId, title, genres	One row per movie. The title ends with the release year in parentheses; genres is a pipe-separated list, e.g. Comedy Drama Romance.
ratings.csv	100,836	userId, movieId, rating, timestamp	One row per rating on a 5-star scale with half-star increments (0.5 – 5.0). timestamp is seconds since the Unix epoch.
tags.csv	3,683	userId, movieId, tag, timestamp	User-generated free-text labels applied to movies.
links.csv	9,742	movieId, imdbId, tmdbId	Identifiers that link each movie to IMDb and The Movie Database.

Statistic	Value
Movies	9,742

Ratings	100,836
Tag applications	3,683
Users	610 (each with ≥ 20 ratings)
Distinct genres	20 (including '(no genres listed)')
Rating scale	0.5 to 5.0 in steps of 0.5
Time span	1996 – 2018
Release years covered	1902 – 2018

Sample rows from the CSV files:

```

movies.csv
movieId,title,genres
1,Toy Story (1995),Adventure|Animation|Children|Comedy|Fantasy
2,Jumanji (1995),Adventure|Children|Fantasy

```

```

ratings.csv
userId,movieId,rating,timestamp
1,1,4.0,964982703
1,3,4.0,964981247

```

```

tags.csv
userId,movieId,tag,timestamp
2,60756,funny,1445714994
2,60756,Highly quotable,1445714996

```

```

links.csv
movieId,imdbId,tmdbId
1,0114709,862

```

6. Why NoSQL / MongoDB for this Problem

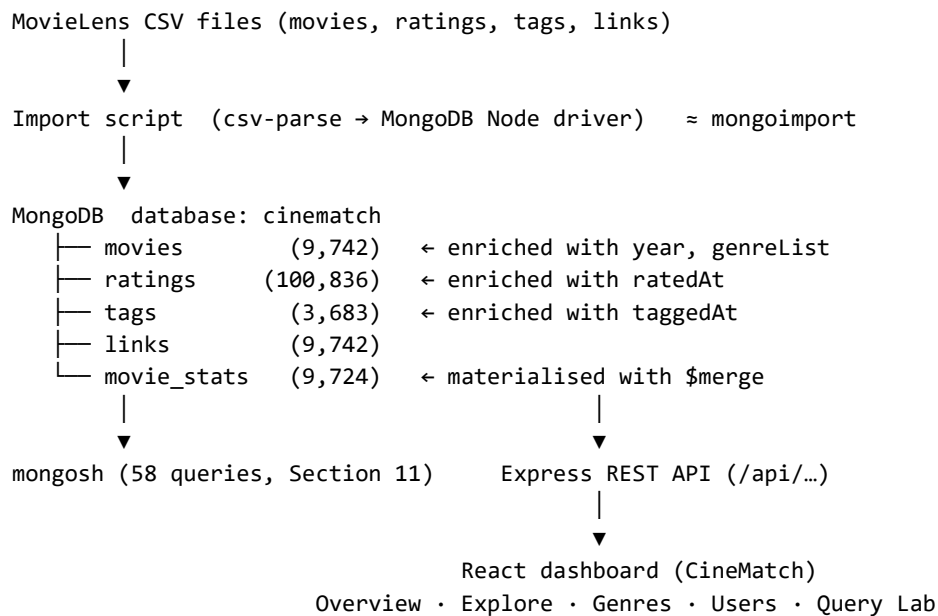
- **Flexible document model:** a movie's genres are a multi-valued attribute. In a relational design this needs a separate movie_genres table and a join; in MongoDB it is simply an array field (genreList) inside the movie document, and a multikey index makes it queryable.
- **In-place enrichment:** pipeline-style updates allowed the year, genre array and ISODate fields to be derived from the raw CSV columns inside the database, without an external ETL tool.
- **Aggregation framework:** multi-stage pipelines (\$match → \$group → \$sort → \$lookup → \$project) express complex analytics such as "best movie per genre" or "users who liked X also liked" in a single query.
- **References and embedding:** ratings reference movies by movieId (a 100k-row fact collection should not be embedded), while derived statistics are materialised into a separate movie_stats collection with \$merge – demonstrating both patterns.
- **Rich operator set:** \$stdDevPop, \$setIntersection, \$facet, \$bucket, \$regexFind and text search cover statistical, set-based, multi-view, histogram, string and search requirements without additional software.
- **Scalability:** the same design would scale horizontally through sharding on movieId/userId when the full 33-million-rating MovieLens dataset is used.

7. Technology Stack

Component	Technology
-----------	------------

Database	MongoDB Community Server 8.3
Shell	mongosh 2.x (all queries in Section 11 were executed here)
Data import	Node.js import script (csv-parse + MongoDB Node driver) – equivalent to <code>mongoimport --type csv --headerline</code>
Backend API	Node.js 24, Express 5, MongoDB Node driver 6
Frontend	React 19 (Vite), React Router, Recharts
Report screenshots	mongosh output captured from the terminal
Operating system	Windows 11

8. System Architecture



The CSV files are imported as-is into four collections. Cleaning queries (Section 11.2) enrich the documents in place. The same aggregation pipelines are then executed in two ways: interactively in `mongosh` for this report, and through an Express API that powers the React dashboard (Section 14).

9. MongoDB Database Design

9.1 Collections

Collection	Documents	Purpose	Key fields
<code>movies</code>	9,742	Movie catalogue	<code>movieId</code> , <code>title</code> , <code>genres</code> , <code>year*</code> , <code>genreList*</code>
<code>ratings</code>	100,836	User ratings (fact collection)	<code>userId</code> , <code>movieId</code> , <code>rating</code> , <code>timestamp</code> , <code>ratedAt*</code>
<code>tags</code>	3,683	Free-text tags	<code>userId</code> , <code>movieId</code> , <code>tag</code> , <code>timestamp</code> , <code>taggedAt*</code>
<code>links</code>	9,742	External identifiers	<code>movieId</code> , <code>imdbId</code> , <code>tmdbId</code>
<code>movie_stats</code>	9,724	Materialised per-movie statistics	<code>_id</code> (<code>movieId</code>), <code>ratingCount</code> , <code>avgRating</code> , <code>stdDev</code>

* fields added by the enrichment queries in Section 11.2.

9.2 Data Model

movies

Field	Type	Description
_id	ObjectId	Generated by MongoDB
movieId	Number	Unique movie identifier (referenced by ratings, tags, links)
title	String	Title with release year, e.g. "Toy Story (1995)"
genres	String	Original pipe-delimited genre string
year	Number	Release year extracted from title (null if absent)
genreList	Array<String>	genres split into an array – multikey indexed

ratings

Field	Type	Description
userId	Number	Anonymous user identifier
movieId	Number	Reference to movies.movieId
rating	Number	0.5 – 5.0
timestamp	Number	Unix epoch seconds (original)
ratedAt	Date	ISODate derived from timestamp

tags

Field	Type	Description
userId	Number	User who applied the tag
movieId	Number	Reference to movies.movieId
tag	String	Free text, e.g. "funny", "Tarantino"
timestamp	Number	Unix epoch seconds
taggedAt	Date	ISODate derived from timestamp

links

Field	Type	Description
movieId	Number	Reference to movies.movieId
imdbId	String	IMDb id (kept as string to preserve leading zeros)
tmdbId	Number	TMDb id (absent for a few movies)

9.3 Relationships

- ratings.movieId → movies.movieId (many-to-one, joined with \$lookup)
- tags.movieId → movies.movieId (many-to-one)
- links.movieId → movies.movieId (one-to-one)
- movie_stats._id → movies.movieId (one-to-one, materialised by \$merge)
- ratings.userId / tags.userId identify users; there is no separate users collection because the dataset contains no user attributes – user profiles are derived by aggregation.

References were preferred over embedding for ratings because a popular movie has hundreds of ratings and the ratings collection is queried independently (by user, by time). Genres, in contrast, are embedded as an array because they are small, bounded and always read together with the movie.

9.4 Sample Documents (after enrichment)

```
// movies
{
  _id: ObjectId('...'),
  movieId: 1,
  title: 'Toy Story (1995)',
  genres: 'Adventure|Animation|Children|Comedy|Fantasy',
  year: 1995,
  genreList: [ 'Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy' ]
}

// ratings
{ userId: 1, movieId: 1, rating: 4, timestamp: 964982703, ratedAt: ISODate('2000-07-30T18:45:03.000Z') }

// tags
{ userId: 2, movieId: 60756, tag: 'funny', timestamp: 1445714994, taggedAt: ISODate('2015-10-24T19:29:54.000Z') }

// movie_stats (materialised)
{ _id: 356, ratingCount: 329, avgRating: 4.164, stdDev: 0.905 }
```

9.5 Indexes

Collection	Index	Type	Purpose
ratings	{ movieId: 1 }	Single field	\$lookup from movies, per-movie statistics
ratings	{ userId: 1 }	Single field	User profiles and collaborative filtering
tags	{ movieId: 1 }	Single field	Tags of a movie
movies	{ movieId: 1 }, unique	Unique	Primary lookup key; prevents duplicate movies
movies	{ genreList: 1 }	Multikey	Genre filters (\$in, \$all) on the array
movies	{ year: 1 }	Single field	Year / decade filters
movies	{ title: 'text' }	Text	Keyword search with \$text

10. Methodology / Implementation

11. **Data acquisition** – the ml-latest-small archive was downloaded and the four CSV files were verified (row counts and headers).
12. **Import** – each CSV was loaded into its own collection in the cinematch database with numeric columns typed as numbers (equivalent to `mongoimport --type csv --headerline`).
13. **Verification** – collection names, document counts and sample documents were checked in mongosh (Queries 1–6).
14. **Cleaning & enrichment** – year, genreList, ratedAt and taggedAt were derived with updateMany pipelines (Queries 7–11).
15. **Indexing** – seven indexes were created on join keys, the genre array and the title (Query 12).
16. **Querying** – basic find/countDocuments/distinct queries with filtering, projection, regex and text search (Queries 13–21).
17. **Aggregation** – rating, genre, user and tag analytics with \$group, \$lookup, \$unwind, \$facet, \$bucket and date operators (Queries 22–48).

18. **Recommendation** – content-based, collaborative and weighted-rating pipelines, plus \$merge to materialise movie_stats (Queries 49–54).
19. **CRUD & performance** – insert/update/delete demonstration and explain() verification of index usage (Queries 55–58).
20. **Dashboard** – the pipelines were wrapped in an Express REST API and visualised in a React application (Section 14).

Every query below was executed in mongosh against the live database; the screenshot under each query is its actual output. Long cursor outputs are truncated in the screenshot where indicated.

11. Queries and Results

The queries are organised into nine groups. For each query the purpose, the exact mongosh command, its output and the observation drawn from the result are given.

11.1 Database Overview

Query 1: List all collections in the cinematch database

Purpose: Verify that the four CSV files were imported as separate collections.

Query:

```
db.getCollectionNames()
```

Output:

```
mongosh - cinematch

cinematch> db.getCollectionNames()
[ 'ratings', 'movie_stats', 'movies', 'links', 'tags' ]
cinematch>
```

Observation: The four CSV files are present as four collections (movie_stats is added later by Query 53).

Query 2: Count documents in every collection

Purpose: Confirm the row counts match the dataset description (9,742 movies, 100,836 ratings, 3,683 tags, 9,742 links).

Query:

```
({
  movies: db.movies.countDocuments(),
  ratings: db.ratings.countDocuments(),
  tags: db.tags.countDocuments(),
  links: db.links.countDocuments()
})
```

Output:

```
mongosh - cinematch

cinematch> ({
...   movies: db.movies.countDocuments(),
...   ratings: db.ratings.countDocuments(),
...   tags: db.tags.countDocuments(),
...   links: db.links.countDocuments()
... })
{ movies: 9742, ratings: 100836, tags: 3683, links: 9742 }
cinematch>
```

Observation: Document counts match the dataset description exactly: 9,742 movies, 100,836 ratings, 3,683 tags and 9,742 links – the import lost no rows.

Query 3: Display a sample movie document

Purpose: Examine the raw document structure produced by the CSV import (genres is a pipe-delimited string).

Query:

```
db.movies.findOne({ movieId: 1 })
```

Output:

```
mongosh - cinematch

cinematch> db.movies.findOne({ movieId: 1 })

{
  _id: ObjectId('6aa6fbf4f05594b31fa64e5c'),
  movieId: 1,
  title: 'Toy Story (1995)',
  genres: 'Adventure|Animation|Children|Comedy|Fantasy'
}

cinematch> 
```

Observation: The raw movie document shows genres as a single pipe-delimited string, which motivates the \$split enrichment in Query 8.

Query 4: Display a sample rating document

Purpose: Ratings reference movies through movieId; timestamp is Unix epoch seconds.

Query:

```
db.ratings.findOne({ userId: 1, movieId: 1 })
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.findOne({ userId: 1, movieId: 1 })

{
  _id: ObjectId('6aa6fbf5f05594b31fa6746a'),
  userId: 1,
  movieId: 1,
  rating: 4,
  timestamp: 964982703
}

cinematch> 
```

Observation: rating and timestamp were imported as numbers; the timestamp (964982703) is Unix epoch seconds and is converted to a Date in Query 9.

Query 5: Display a sample tag and link document

Purpose: Tags are free-text labels applied by users; links map movieId to IMDb and TMDb identifiers.

Query:

```
[ db.tags.findOne({ movieId: 60756 }), db.links.findOne({ movieId: 1 }) ]
```

Output:

```
mongosh - cinematch

cinematch> [ db.tags.findOne({ movieId: 60756 }), db.links.findOne({ movieId: 1 }) ]

[
  {
    _id: ObjectId('6aa6fbf6f05594b31fa7fe4e'),
    userId: 2,
    movieId: 60756,
    tag: 'funny',
    timestamp: 1445714994
  },
  {
    _id: ObjectId('6aa6fbf6f05594b31fa80cb1'),
    movieId: 1,
    imdbId: '0114709',
    tmdbId: 862
  }
]

cinematch>
```

Observation: Tags are free text with arbitrary casing; imdbId is stored as a string to preserve its leading zero.

Query 6: Count the number of distinct users who rated movies

Purpose: Establish the size of the user base (610 users, each with at least 20 ratings).

Query:

```
db.ratings.aggregate([ { $group: { _id: "$userId" } }, { $count: "totalUsers" } ])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([ { $group: { _id: "$userId" } }, { $count: "totalUsers" } ])

[ { totalUsers: 610 } ]

cinematch>
```

Observation: 610 distinct users produced the 100,836 ratings – an average of about 165 ratings per user.

11.2 Data Cleaning & Schema Enrichment

Query 7: Extract the release year from the movie title

Purpose: Titles end with the year in parentheses, e.g. 'Toy Story (1995)'. A pipeline-style update with \$regexFind captures it and \$toInt stores it as a numeric year field (null when a title has no year).

Query:

```
db.movies.updateMany({}, [
  {
    $set: {
      year: {
        $let: {
          vars: { m: { $regexFind: { input: "$title", regex: /\((\d{4})\)\s*$/ } } },
          in: {
            $cond: [
              { $eq: ["$$m", null] },
              null,
              { $toInt: { $arrayElemAt: ["$$m.captures", 0] } }
            ]
          }
        }
      }
    }
  }
])
```

```

    }
  }
})

```

Output:

```

mongosh - cinematch

cinematch> db.movies.updateMany({}, [
...   {
...     $set: {
...       year: {
...         $let: {
...           vars: { m: { $regexFind: { input: "$title", regex: /\((\d{4})\)\s*$/ } } },
...           in: {
...             $cond: [
...               { $eq: ["$$m", null] },
...               null,
...               { $toInt: { $arrayElemAt: ["$$m.captures", 0] } }
...             ]
...           }
...         }
...       }
...     }
...   }
... ])

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9742,
  modifiedCount: 9742,
  upsertedCount: 0
}

cinematch>

```

Observation: All 9,742 movie documents were updated; the year is now a numeric field that can be filtered, sorted and bucketed.

Query 8: Split the pipe-delimited genres string into an array

Purpose: genres: 'Adventure|Animation|Children' becomes genreList: ['Adventure','Animation','Children'] so genres can be queried and unwound individually.

Query:

```

db.movies.updateMany({}, [
  { $set: { genreList: { $split: ["$genres", "|"] } } }
])

```

Output:

```

mongosh - cinematch

cinematch> db.movies.updateMany({}, [
...   { $set: { genreList: { $split: ["$genres", "|"] } } }
... ])

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9742,
  modifiedCount: 9742,
  upsertedCount: 0
}

cinematch>

```

Observation: genreList is now an array, so a movie with three genres can be matched on any one of them and unwound for per-genre statistics.

Query 9: Convert Unix timestamps into ISODate fields

Purpose: Epoch seconds are multiplied by 1000 and converted with \$toDate so that \$year, \$month and \$dayOfWeek can be used later.

Query:

```
db.ratings.updateMany({}, [
  { $set: { ratedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
]);
db.tags.updateMany({}, [
  { $set: { taggedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.updateMany({}, [
...   { $set: { ratedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
...   ]);
...   db.tags.updateMany({}, [
...     { $set: { taggedAt: { $toDate: { $multiply: ["$timestamp", 1000] } } } }
...   ])
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3683,
  modifiedCount: 3683,
  upsertedCount: 0
}
cinematch>
```

Observation: 100,836 ratings and 3,683 tags now carry a proper ISODate, enabling \$year, \$dayOfWeek and date range queries.

Query 10: Normalise titles that end with a trailing article

Purpose: MovieLens stores titles as 'Shawshank Redemption, The (1994)'. A \$regexFind update moves the trailing article (The/A/An) back to the front so titles read naturally in results.

Query:

```
db.movies.updateMany(
  { title: /, (The|A|An) \(/ },
  [
    {
      $set: {
        title: {
          $let: {
            vars: { m: { $regexFind: { input: "$title", regex: /^(.+), (The|A|An) (\(.+)\$/ }
          } },
            in: {
              $concat: [
                { $arrayElemAt: ["$$m.captures", 1] }, " ",
                { $arrayElemAt: ["$$m.captures", 0] }, " ",
                { $arrayElemAt: ["$$m.captures", 2] }
              ]
            }
          }
        }
      }
    }
  ]
)
```

```
]
)
```

Output:

```
mongosh - cinematch

cinematch> db.movies.updateMany(
...   { title: /, (The|A|An) \(/ },
...   [
...     {
...       $set: {
...         title: {
...           $let: {
...             vars: { m: { $regexFind: { input: "$title", regex: /^(.+), (The|A|An) (\(.+\)$/ } } },
...             in: {
...               $concat: [
...                 { $arrayElemAt: [ "$$m.captures", 1 ] }, " ",
...                 { $arrayElemAt: [ "$$m.captures", 0 ] }, " ",
...                 { $arrayElemAt: [ "$$m.captures", 2 ] }
...               ]
...             }
...           }
...         }
...       }
...     }
...   ]
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1822,
  modifiedCount: 1822,
  upsertedCount: 0
}

cinematch> 
```

Observation: 1,822 titles matched the pattern and were rewritten – e.g. 'Shawshank Redemption, The (1994)' is now 'The Shawshank Redemption (1994)'. All later results use the readable form.

Query 11: Verify the enriched movie documents

Purpose: Toy Story now carries year and genreList in addition to the original fields, and movieId 318 reads 'The Shawshank Redemption (1994)' after title normalisation.

Query:

```
[ db.movies.findOne({ movieId: 1 }), db.movies.findOne({ movieId: 318 }, { _id: 0, title: 1,
year: 1, genreList: 1 }) ]
```

Output:

```
mongosh - cinematch

cinematch> [ db.movies.findOne({ movieId: 1 }), db.movies.findOne({ movieId: 318 }, { _id: 0, title: 1, year: 1, genreList:
...      }) ]

[
  {
    _id: ObjectId('6aa6fbf4f05594b31fa64e5c'),
    movieId: 1,
    title: 'Toy Story (1995)',
    genres: 'Adventure|Animation|Children|Comedy|Fantasy',
    year: 1995,
    genreList: [ 'Adventure', 'Animation', 'Children', 'Comedy', 'Fantasy' ]
  },
  {
    title: 'The Shawshank Redemption (1994)',
    year: 1994,
    genreList: [ 'Crime', 'Drama' ]
  }
]

cinematch> 
```

Observation: Toy Story now has year: 1995 and a five-element genreList, and movieId 318 shows the normalised title – schema evolution without any migration script.

Query 12: Create indexes on frequently queried fields

Purpose: movieId/userId support \$lookup joins, genreList becomes a multikey index, and a text index enables keyword search on titles.

Query:

```
db.ratings.createIndex({ movieId: 1 });
db.ratings.createIndex({ userId: 1 });
db.tags.createIndex({ movieId: 1 });
db.movies.createIndex({ movieId: 1 }, { unique: true });
db.movies.createIndex({ genreList: 1 });
db.movies.createIndex({ year: 1 });
db.movies.createIndex({ title: "text" });
db.movies.getIndexes().map(i => i.name)
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.createIndex({ movieId: 1 });
...      db.ratings.createIndex({ userId: 1 });
...      db.tags.createIndex({ movieId: 1 });
...      db.movies.createIndex({ movieId: 1 }, { unique: true });
...      db.movies.createIndex({ genreList: 1 });
...      db.movies.createIndex({ year: 1 });
...      db.movies.createIndex({ title: "text" });
...      db.movies.getIndexes().map(i => i.name)

[ '_id_', 'movieId_1', 'genreList_1', 'year_1', 'title_text' ]

cinematch> 
```

Observation: Seven indexes exist on the movies collection alone (including the default _id index); the multikey index on genreList and the text index on title are used by later queries.

11.3 Movie Exploration

Query 13: Find movies released in 1995

Purpose: Simple equality filter with projection to hide _id and show only the title and genres.

Query:

```
db.movies.find({ year: 1995 }, { _id: 0, movieId: 1, title: 1, genres: 1 }).limit(10)
```

Output:

```
mongosh - cinematch

cinematch> db.movies.find({ year: 1995 }, { _id: 0, movieId: 1, title: 1, genres: 1 }).limit(10)

[
  {
    movieId: 1,
    title: 'Toy Story (1995)',
    genres: 'Adventure|Animation|Children|Comedy|Fantasy'
  },
  {
    movieId: 2,
    title: 'Jumanji (1995)',
    genres: 'Adventure|Children|Fantasy'
  },
  {
    movieId: 3,
    title: 'Grumpier Old Men (1995)',
    genres: 'Comedy|Romance'
  },
  {
    movieId: 4,
    title: 'Waiting to Exhale (1995)',
    genres: 'Comedy|Drama|Romance'
  },
  {
    movieId: 5,
    title: 'Father of the Bride Part II (1995)',
    genres: 'Comedy'
  },
  { movieId: 6, title: 'Heat (1995)', genres: 'Action|Crime|Thriller' },
  { movieId: 7, title: 'Sabrina (1995)', genres: 'Comedy|Romance' },
  {
    movieId: 8,
    title: 'Tom and Huck (1995)',
    genres: 'Adventure|Children'
  },
  { movieId: 9, title: 'Sudden Death (1995)', genres: 'Action' },
  {
    movieId: 10,
    title: 'GoldenEye (1995)',
    genres: 'Action|Adventure|Thriller'
  }
]

cinematch> █
```

Observation: 1995 has 259 movies in the catalogue; the projection returns only movieId, title and genres and .limit(10) keeps the output short.

Query 14: Search movie titles with a regular expression

Purpose: Case-insensitive pattern matching to find every Star Wars film in the catalogue.

Query:

```
db.movies.find({ title: { $regex: "star wars", $options: "i" } }, { _id: 0, title: 1, year: 1
}).sort({ year: 1 })
```

Output:

```
mongosh - cinematch

cinematch> db.movies.find({ title: { $regex: "star wars", $options: "i" } }, { _id: 0, title: 1, year: 1 }).sort({ year: 1 })

[
  { title: 'Star Wars: Episode IV - A New Hope (1977)', year: 1977 },
  { title: 'The Star Wars Holiday Special (1978)', year: 1978 },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    year: 1980
  },
  {
    title: 'Star Wars: Episode VI - Return of the Jedi (1983)',
    year: 1983
  },
  {
    title: 'Star Wars: Episode I - The Phantom Menace (1999)',
    year: 1999
  },
  {
    title: 'Star Wars: Episode II - Attack of the Clones (2002)',
    year: 2002
  },
  {
    title: 'Empire of Dreams: The Story of the 'Star Wars' Trilogy (2004)',
    year: 2004
  },
  {
    title: 'Star Wars: Episode III - Revenge of the Sith (2005)',
    year: 2005
  },
  { title: 'Star Wars: The Clone Wars (2008)', year: 2008 },
  {
    title: 'Star Wars: Episode VII - The Force Awakens (2015)',
    year: 2015
  },
  { title: 'Rogue One: A Star Wars Story (2016)', year: 2016 },
  { title: 'Star Wars: The Last Jedi (2017)', year: 2017 },
  { title: 'Solo: A Star Wars Story (2018)', year: 2018 }
]

cinematch> 
```

Observation: The case-insensitive regex returns all 13 titles containing 'star wars' in chronological order – the nine saga films plus spin-offs such as Rogue One, Solo and the Holiday Special.

Query 15: Find movies that are both Comedy and Romance

Purpose: \$all matches documents whose genreList array contains every listed genre.

Query:

```
db.movies.find({ genreList: { $all: ["Comedy", "Romance"] }, year: { $gte: 2015 } }, { _id: 0, title: 1, genres: 1 }).limit(10)
```

Output:

```
mongosh - cinematch

cinematch> db.movies.find({ genreList: { $all: ["Comedy", "Romance"] }, year: { $gte: 2015 } }, { _id: 0, title: 1, genres:
...    }).limit(10)

[
  {
    title: 'Mortdecai (2015)',
    genres: 'Adventure|Comedy|Mystery|Romance'
  },
  { title: 'Boy Meets Girl (2015)', genres: 'Comedy|Drama|Romance' },
  { title: 'Focus (2015)', genres: 'Comedy|Crime|Drama|Romance' },
  { title: 'Man Up (2015)', genres: 'Comedy|Romance' },
  { title: 'The Lobster (2015)', genres: 'Comedy|Romance|Sci-Fi' },
  { title: 'Trainwreck (2015)', genres: 'Comedy|Romance' },
  { title: 'Aloha (2015)', genres: 'Comedy|Drama|Romance' },
  { title: 'The Escort (2015)', genres: 'Comedy|Romance' },
  { title: 'Slow Learners (2015)', genres: 'Comedy|Romance' },
  { title: 'How to Be Single (2016)', genres: 'Comedy|Romance' }
]

cinematch>
```

Observation: \$all requires both genres to be present in the array; the query returns recent romantic comedies only.

Query 16: Count movies with no genres listed

Purpose: Data-quality check: some entries carry the placeholder '(no genres listed)'.

Query:

```
db.movies.countDocuments({ genres: "(no genres listed)" })
```

Output:

```
mongosh - cinematch

cinematch> db.movies.countDocuments({ genres: "(no genres listed)" })

34

cinematch>
```

Observation: 34 movies carry the placeholder '(no genres listed)'; they are excluded from genre analytics.

Query 17: Number of movies per genre

Purpose: \$unwind turns each genre into a separate document so \$group can count them.

Query:

```
db.movies.aggregate([
  { $unwind: "$genreList" },
  { $group: { _id: "$genreList", movieCount: { $sum: 1 } } },
  { $sort: { movieCount: -1 } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...     { $unwind: "$genreList" },
...     { $group: { _id: "$genreList", movieCount: { $sum: 1 } } },
...     { $sort: { movieCount: -1 } }
... ])

[
  { _id: 'Drama', movieCount: 4361 },
  { _id: 'Comedy', movieCount: 3756 },
  { _id: 'Thriller', movieCount: 1894 },
  { _id: 'Action', movieCount: 1828 },
  { _id: 'Romance', movieCount: 1596 },
  { _id: 'Adventure', movieCount: 1263 },
  { _id: 'Crime', movieCount: 1199 },
  { _id: 'Sci-Fi', movieCount: 980 },
  { _id: 'Horror', movieCount: 978 },
  { _id: 'Fantasy', movieCount: 779 },
  { _id: 'Children', movieCount: 664 },
  { _id: 'Animation', movieCount: 611 },
  { _id: 'Mystery', movieCount: 573 },
  { _id: 'Documentary', movieCount: 440 },
  { _id: 'War', movieCount: 382 },
  { _id: 'Musical', movieCount: 334 },
  { _id: 'Western', movieCount: 167 },
  { _id: 'IMAX', movieCount: 158 },
  { _id: 'Film-Noir', movieCount: 87 },
  { _id: '(no genres listed)', movieCount: 34 }
]

cinematch> 
```

Observation: Drama (4,361) and Comedy (3,756) dominate the catalogue, followed by Thriller, Action and Romance; Film-Noir (87) is the rarest genre.

Query 18: Number of movies released per decade

Purpose: Bucket movies by decade using arithmetic on the year field.

Query:

```
db.movies.aggregate([
  { $match: { year: { $ne: null } } },
  {
    $group: {
      _id: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
      movies: { $sum: 1 }
    }
  },
  { $sort: { _id: 1 } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...     { $match: { year: { $ne: null } } },
...     {
...         $group: {
...             _id: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
...             movies: { $sum: 1 }
...         }
...     },
...     { $sort: { _id: 1 } }
... ])

[
  { _id: 1900, movies: 3 },
  { _id: 1910, movies: 7 },
  { _id: 1920, movies: 37 },
  { _id: 1930, movies: 136 },
  { _id: 1940, movies: 197 },
  { _id: 1950, movies: 279 },
  { _id: 1960, movies: 401 },
  { _id: 1970, movies: 500 },
  { _id: 1980, movies: 1177 },
  { _id: 1990, movies: 2212 },
  { _id: 2000, movies: 2849 },
  { _id: 2010, movies: 1931 }
]

cinematch>
```

Observation: The 1990s and 2000s each contain over 2,000 movies; the catalogue thins rapidly before 1950 and after 2015 (dataset generated in 2018).

Query 19: Top 10 years with the most movie releases

Purpose: Group by year, count and rank.

Query:

```
db.movies.aggregate([
  { $match: { year: { $ne: null } } },
  { $group: { _id: "$year", movies: { $sum: 1 } } },
  { $sort: { movies: -1 } },
  { $limit: 10 }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...     { $match: { year: { $ne: null } } },
...     { $group: { _id: "$year", movies: { $sum: 1 } } },
...     { $sort: { movies: -1 } },
...     { $limit: 10 }
... ])

[
  { _id: 2002, movies: 311 },
  { _id: 2006, movies: 295 },
  { _id: 2001, movies: 294 },
  { _id: 2007, movies: 284 },
  { _id: 2000, movies: 283 },
  { _id: 2009, movies: 282 },
  { _id: 2004, movies: 279 },
  { _id: 2003, movies: 279 },
  { _id: 2014, movies: 278 },
  { _id: 1996, movies: 276 }
]

cinematch>
```

Observation: 2002 (311 movies) and 2006 (295) are the most represented years, followed by 2001, 2007 and 2000.

Query 20: Most common genre combinations

Purpose: Grouping on the original genres string reveals which multi-genre combinations occur most often.

Query:

```
db.movies.aggregate([
  { $match: { genreList: { $not: { $size: 1 } } } },
  { $group: { _id: "$genres", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 10 }
])
```

Output:



The screenshot shows a terminal window titled 'mongosh - cinematch'. The user enters the aggregation query, and the output is displayed as a JSON array of objects. Each object contains an '_id' representing a genre combination and a 'count' representing the number of movies in that category. The results are sorted in descending order of count.

```
cinematch> db.movies.aggregate([
...   { $match: { genreList: { $not: { $size: 1 } } } },
...   { $group: { _id: "$genres", count: { $sum: 1 } } },
...   { $sort: { count: -1 } },
...   { $limit: 10 }
... ])
[
  { _id: 'Comedy|Drama', count: 435 },
  { _id: 'Comedy|Romance', count: 363 },
  { _id: 'Drama|Romance', count: 349 },
  { _id: 'Comedy|Drama|Romance', count: 276 },
  { _id: 'Drama|Thriller', count: 168 },
  { _id: 'Horror|Thriller', count: 135 },
  { _id: 'Crime|Drama', count: 134 },
  { _id: 'Crime|Drama|Thriller', count: 125 },
  { _id: 'Drama|War', count: 114 },
  { _id: 'Comedy|Crime', count: 101 }
]
cinematch> █
```

Observation: Comedy|Drama, Comedy|Romance and Drama|Romance are the most frequent multi-genre combinations – romantic comedies and dramedies dominate the catalogue.

Query 21: Full-text search on movie titles

Purpose: Uses the text index created earlier; \$meta returns the relevance score.

Query:

```
db.movies.find({ $text: { $search: "lord rings" } }, { _id: 0, title: 1, score: { $meta:
"textScore" } }).sort({ score: { $meta: "textScore" } }).limit(5)
```

Output:

```
mongosh - cinematch

cinematch> db.movies.find({ $text: { $search: "lord rings" } }, { _id: 0, title: 1, score: { $meta: "textScore" } }).sort({
...     score: { $meta: "textScore" } }).limit(5)

[
  {
    title: 'The Lord of the Rings: The Fellowship of the Ring (2001)',
    score: 1.65
  },
  { title: 'The Lord of the Rings (1978)', score: 1.3333333333333333 },
  {
    title: 'The Lord of the Rings: The Return of the King (2003)',
    score: 1.2
  },
  { title: 'The Lord of the Rings: The Two Towers (2002)', score: 1.2 },
  { title: 'The Ring (2002)', score: 0.75 }
]

cinematch>
```

Observation: The text index ranks the Lord of the Rings trilogy first; textScore reflects term frequency in the title.

11.4 Rating Analysis

Query 22: Rating distribution (how often each star value is used)

Purpose: Ratings run from 0.5 to 5.0 in half-star steps; the distribution shows users favour whole stars and 4.0 is the most common value.

Query:

```
db.ratings.aggregate([
  { $group: { _id: "$rating", count: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...     { $group: { _id: "$rating", count: { $sum: 1 } } },
...     { $sort: { _id: 1 } }
...     ])

[
  { _id: 0.5, count: 1370 },
  { _id: 1, count: 2811 },
  { _id: 1.5, count: 1791 },
  { _id: 2, count: 7551 },
  { _id: 2.5, count: 5550 },
  { _id: 3, count: 20047 },
  { _id: 3.5, count: 13136 },
  { _id: 4, count: 26818 },
  { _id: 4.5, count: 8551 },
  { _id: 5, count: 13211 }
]

cinematch>
```

Observation: 4.0 is by far the most common rating (26,818), followed by 3.0 and 5.0. Whole-star values are used far more than half-stars, showing users round their opinions.

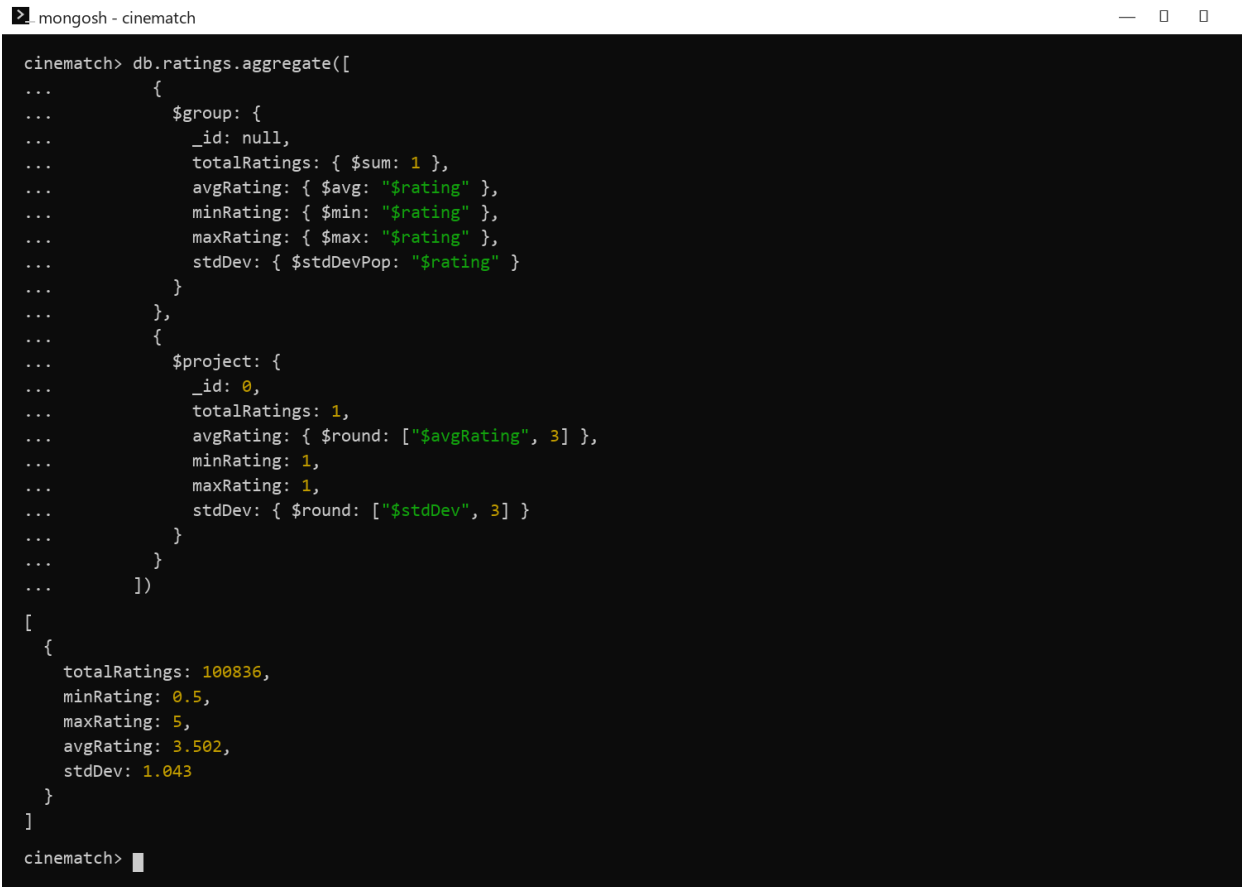
Query 23: Overall rating statistics

Purpose: \$avg, \$min, \$max and \$stdDevPop over the entire ratings collection.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: null,
      totalRatings: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      minRating: { $min: "$rating" },
      maxRating: { $max: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  {
    $project: {
      _id: 0,
      totalRatings: 1,
      avgRating: { $round: ["$avgRating", 3] },
      minRating: 1,
      maxRating: 1,
      stdDev: { $round: ["$stdDev", 3] }
    }
  }
])
```

Output:



```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: null,
...       totalRatings: { $sum: 1 },
...       avgRating: { $avg: "$rating" },
...       minRating: { $min: "$rating" },
...       maxRating: { $max: "$rating" },
...       stdDev: { $stdDevPop: "$rating" }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       totalRatings: 1,
...       avgRating: { $round: ["$avgRating", 3] },
...       minRating: 1,
...       maxRating: 1,
...       stdDev: { $round: ["$stdDev", 3] }
...     }
...   }
... ])

[
  {
    totalRatings: 100836,
    minRating: 0.5,
    maxRating: 5,
    avgRating: 3.502,
    stdDev: 1.043
  }
]

cinematch> 
```

Observation: The global mean is 3.50 with a standard deviation of 1.04; this value of 3.5 is used as the prior C in the weighted-rating formula (Query 31).

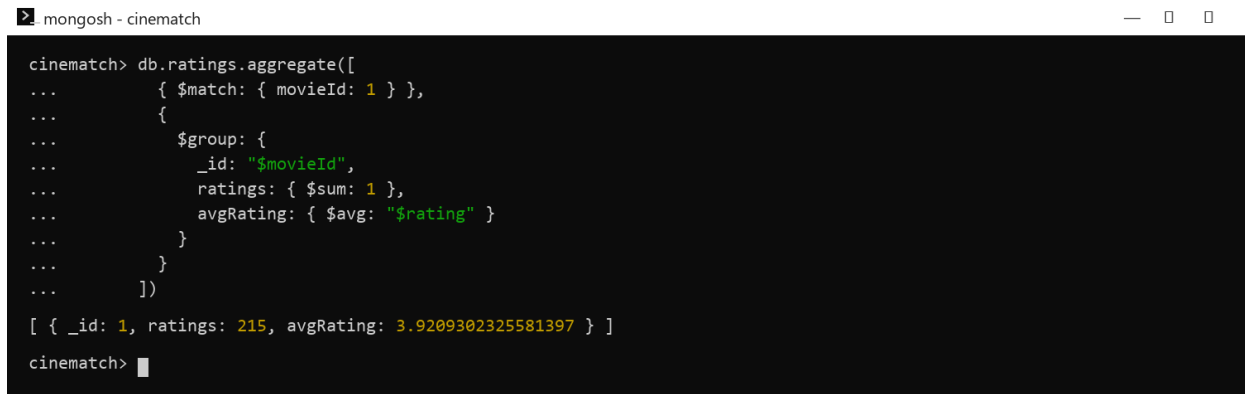
Query 24: Average rating of a specific movie (Toy Story)

Purpose: \$match narrows to one movieId before grouping.

Query:

```
db.ratings.aggregate([
  { $match: { movieId: 1 } },
  {
    $group: {
      _id: "$movieId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  }
])
```

Output:



The screenshot shows a terminal window titled 'mongosh - cinematch'. The user enters the following aggregation query:

```
cinematch> db.ratings.aggregate([
...     { $match: { movieId: 1 } },
...     {
...       $group: {
...         _id: "$movieId",
...         ratings: { $sum: 1 },
...         avgRating: { $avg: "$rating" }
...       }
...     }
...   ])
```

The terminal displays the result as a JSON array:

```
[ { _id: 1, ratings: 215, avgRating: 3.9209302325581397 } ]
```

The prompt 'cinematch>' is visible at the bottom of the terminal.

Observation: Toy Story has 215 ratings with an average of 3.92 – a well-liked and widely watched film.

Query 25: Top 10 most-rated (most popular) movies

Purpose: Count ratings per movie, then \$lookup joins the movies collection to attach the title.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $sort: { ratingCount: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

```
}  
])
```

Output:

```
mongosh - cinematch  
  
cinematch> db.ratings.aggregate([  
...   {  
...     $group: {  
...       _id: "$movieId",  
...       ratingCount: { $sum: 1 },  
...       avgRating: { $avg: "$rating" }  
...     }  
...   },  
...   { $sort: { ratingCount: -1 } },  
...   { $limit: 10 },  
...   {  
...     $lookup: {  
...       from: "movies",  
...       localField: "_id",  
...       foreignField: "movieId",  
...       as: "movie"  
...     }  
...   },  
...   { $unwind: "$movie" },  
...   {  
...     $project: {  
...       _id: 0,  
...       title: "$movie.title",  
...       ratingCount: 1,  
...       avgRating: { $round: ["$avgRating", 2] }  
...     }  
...   }  
... ])  
  
[  
  { ratingCount: 329, title: 'Forrest Gump (1994)', avgRating: 4.16 },  
  {  
    ratingCount: 317,  
    title: 'The Shawshank Redemption (1994)',  
    avgRating: 4.43  
  },  
  { ratingCount: 307, title: 'Pulp Fiction (1994)', avgRating: 4.2 },  
  {  
    ratingCount: 279,  
    title: 'The Silence of the Lambs (1991)',  
    avgRating: 4.16  
  },  
  { ratingCount: 278, title: 'The Matrix (1999)', avgRating: 4.19 },  
  {  
    ratingCount: 251,  
    title: 'Star Wars: Episode IV - A New Hope (1977)',  
    avgRating: 4.23  
  },  
  { ratingCount: 238, title: 'Jurassic Park (1993)', avgRating: 3.75 },  
  { ratingCount: 237, title: 'Braveheart (1995)', avgRating: 4.03 },  
  {  
    ratingCount: 224,  
    title: 'Terminator 2: Judgment Day (1991)',  
    avgRating: 3.97  
  },  
  {  
    ratingCount: 220,  
    title: 'Schindler's List (1993)',  
    avgRating: 4.22  
  }  
]  
  
cinematch> 
```

Observation: Forrest Gump (329 ratings), The Shawshank Redemption (317) and Pulp Fiction (307) are the most-rated movies – 1990s classics dominate popularity.

Query 26: Top 10 highest-rated movies (minimum 100 ratings)

Purpose: A minimum rating count avoids movies rated 5.0 by a single user.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 100 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      year: "$movie.year",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     },
...     { $match: { ratingCount: { $gte: 100 } } },
...     { $sort: { avgRating: -1 } },
...     { $limit: 10 },
...     {
...       $lookup: {
...         from: "movies",
...         localField: "_id",
...         foreignField: "movieId",
...         as: "movie"
...       },
...       { $unwind: "$movie" },
...       {
...         $project: {
...           _id: 0,
...           title: "$movie.title",
...           year: "$movie.year",
...           ratingCount: 1,
...           avgRating: { $round: ["$avgRating", 2] }
...         }
...       }
...     }
...   ]
... )

[
  {
    ratingCount: 317,
    title: 'The Shawshank Redemption (1994)',
    year: 1994,
    avgRating: 4.43
  },
  {
    ratingCount: 192,
    title: 'The Godfather (1972)',
    year: 1972,
    avgRating: 4.29
  },
  {
    ratingCount: 218,
    title: 'Fight Club (1999)',
    year: 1999,
    avgRating: 4.27
  },
  {
    ratingCount: 129,
    title: 'The Godfather: Part II (1974)',
    year: 1974,
    avgRating: 4.26
  },
  {
    ratingCount: 107,
    title: 'The Departed (2006)',
    year: 2006,
    avgRating: 4.25
  },
  {
    ratingCount: 126,
    title: 'Goodfellas (1990)',
    year: 1990,
    avgRating: 4.25
  },
  {
    ratingCount: 100,
    title: 'Casablanca (1942)',
    year: 1942,
    avgRating: 4.24
  },
  {
    ratingCount: 149,
    title: 'The Dark Knight (2008)',
    year: 2008,
    avgRating: 4.24
  },
  {
    ratingCount: 204,
    title: 'The Usual Suspects (1995)',
    year: 1995,
    avgRating: 4.24
  },
  {
    ratingCount: 142,
    title: 'The Princess Bride (1987)',
    year: 1987,
    avgRating: 4.23
  }
]

cinematch>
```

Observation: With a 100-rating minimum, The Shawshank Redemption (4.43) leads, followed by The Godfather (4.29) and Fight Club (4.27) – the list closely mirrors the IMDb Top 250.

Query 27: 10 lowest-rated movies (minimum 30 ratings)

Purpose: Same pipeline sorted ascending to find widely watched but poorly received films.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 30 } } },
  { $sort: { avgRating: 1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   { $match: { ratingCount: { $gte: 30 } } },
...   { $sort: { avgRating: 1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])

[
  { ratingCount: 33, title: 'Godzilla (1998)', avgRating: 1.95 },
  {
    ratingCount: 32,
    title: 'I Know What You Did Last Summer (1997)',
    avgRating: 2.11
  },
  { ratingCount: 53, title: 'Wild Wild West (1999)', avgRating: 2.21 },
  { ratingCount: 42, title: 'Batman & Robin (1997)', avgRating: 2.21 },
  { ratingCount: 39, title: 'Hollow Man (2000)', avgRating: 2.29 },
  { ratingCount: 42, title: 'The Flintstones (1994)', avgRating: 2.36 },
  { ratingCount: 37, title: 'Free Willy (1993)', avgRating: 2.39 },
  { ratingCount: 41, title: 'Striptease (1996)', avgRating: 2.41 },
  { ratingCount: 63, title: 'Coneheads (1993)', avgRating: 2.42 },
  { ratingCount: 35, title: 'Showgirls (1995)', avgRating: 2.47 }
]

cinematch> 
```

Observation: Widely watched but poorly received: Godzilla (1998) averages 1.95, followed by I Know What You Did Last Summer, Wild Wild West and Batman & Robin – all below 2.3.

Query 28: Most controversial movies (highest rating variance)

Purpose: \$stdDevPop measures disagreement between users; a high value means the audience is split between loving and hating the film.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
```

```
{ $sort: { stdDev: -1 } },
{ $limit: 10 },
{
  $lookup: {
    from: "movies",
    localField: "_id",
    foreignField: "movieId",
    as: "movie"
  }
},
{ $unwind: "$movie" },
{
  $project: {
    _id: 0,
    title: "$movie.title",
    ratingCount: 1,
    avgRating: { $round: ["$avgRating", 2] },
    stdDev: { $round: ["$stdDev", 3] }
  }
}
])
```

Output:

mongosh - cinematch

```
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" },
...       stdDev: { $stdDevPop: "$rating" }
...     }
...   },
...   { $match: { ratingCount: { $gte: 50 } } },
...   { $sort: { stdDev: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] },
...       stdDev: { $round: ["$stdDev", 3] }
...     }
...   }
... ])
[
  {
    ratingCount: 64,
    title: 'The Blair Witch Project (1999)',
    avgRating: 2.8,
    stdDev: 1.363
  },
  {
    ratingCount: 54,
    title: 'First Knight (1995)',
    avgRating: 3.08,
    stdDev: 1.287
  },
  {
    ratingCount: 121,
    title: 'Austin Powers: The Spy Who Shagged Me (1999)',
    avgRating: 3.2,
    stdDev: 1.21
  },
  {
    ratingCount: 70,
    title: 'Scream (1996)',
    avgRating: 3.2,
    stdDev: 1.187
  },
  {
    ratingCount: 55,
    title: 'Moulin Rouge (2001)',
    avgRating: 3.59,
    stdDev: 1.187
  },
  {
    ratingCount: 86,
    title: 'Mars Attacks! (1996)',
    avgRating: 3.09,
    stdDev: 1.173
  },
  {
    ratingCount: 82,
    title: 'The Nutty Professor (1996)',
    avgRating: 2.73,
    stdDev: 1.167
  },
  {
    ratingCount: 54,
    title: 'The Cable Guy (1996)',
    avgRating: 2.81,
    stdDev: 1.164
  },
  {
    ratingCount: 50,
    title: 'Scary Movie (2000)',
    avgRating: 2.92,
    stdDev: 1.159
  },
  {
    ratingCount: 56,
    title: 'Rushmore (1998)',
    avgRating: 3.54,
    stdDev: 1.147
  }
]
cinematch>
```

Observation: The Blair Witch Project (σ 1.36, mean 2.8) is the most divisive film, followed by First Knight and Austin Powers – audiences split between loving and hating them; the average alone hides this disagreement.

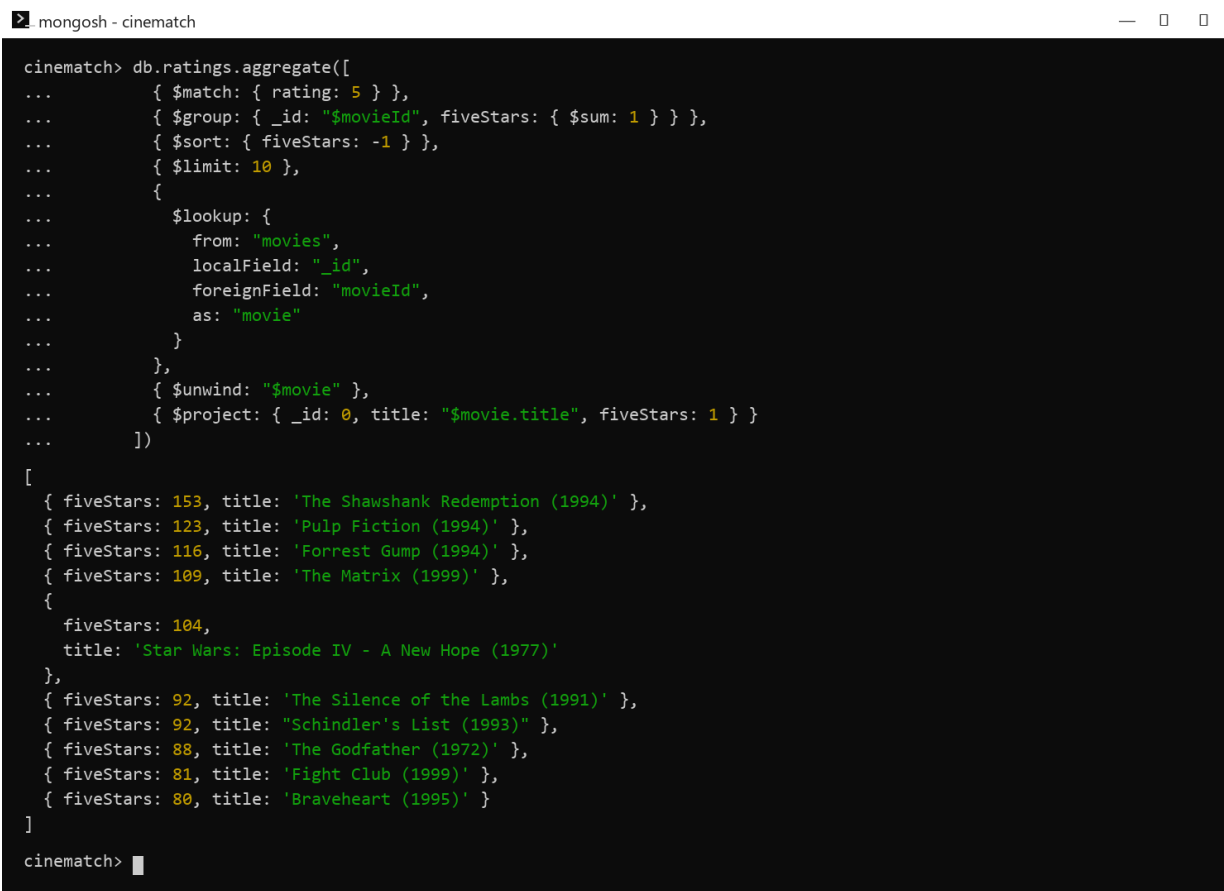
Query 29: Movies with the most 5-star ratings

Purpose: Filter to perfect scores first, then count per movie.

Query:

```
db.ratings.aggregate([
  { $match: { rating: 5 } },
  { $group: { _id: "$movieId", fiveStars: { $sum: 1 } } },
  { $sort: { fiveStars: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $project: { _id: 0, title: "$movie.title", fiveStars: 1 } }
])
```

Output:



The screenshot shows a MongoDB terminal window with the title 'mongosh - cinematch'. The user enters an aggregation query to find movies with the most 5-star ratings. The output is a JSON array of 10 objects, each containing the number of 5-star ratings and the movie title.

```
cinematch> db.ratings.aggregate([
...   { $match: { rating: 5 } },
...   { $group: { _id: "$movieId", fiveStars: { $sum: 1 } } },
...   { $sort: { fiveStars: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   { $project: { _id: 0, title: "$movie.title", fiveStars: 1 } }
... ])

[
  { fiveStars: 153, title: 'The Shawshank Redemption (1994)' },
  { fiveStars: 123, title: 'Pulp Fiction (1994)' },
  { fiveStars: 116, title: 'Forrest Gump (1994)' },
  { fiveStars: 109, title: 'The Matrix (1999)' },
  {
    fiveStars: 104,
    title: 'Star Wars: Episode IV - A New Hope (1977)'
  },
  { fiveStars: 92, title: 'The Silence of the Lambs (1991)' },
  { fiveStars: 92, title: 'Schindler's List (1993)' },
  { fiveStars: 88, title: 'The Godfather (1972)' },
  { fiveStars: 81, title: 'Fight Club (1999)' },
  { fiveStars: 80, title: 'Braveheart (1995)' }
]

cinematch> █
```

Observation: Shawshank Redemption, Pulp Fiction and Forrest Gump have the most perfect scores, combining popularity with quality.

Query 30: Movies that have never been rated

Purpose: A `$lookup` from movies to ratings followed by `$match` on an empty array finds catalogue entries with zero ratings (an anti-join).

Query:

```
db.movies.aggregate([
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $match: { r: { $size: 0 } } },
  { $count: "unratedMovies" }
])
```

Output:



```
cinematch> db.movies.aggregate([
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   { $match: { r: { $size: 0 } } },
...   { $count: "unratedMovies" }
... ])
[ { unratedMovies: 18 } ]
cinematch>
```

Observation: 18 catalogue entries have never been rated – the anti-join pattern (`$lookup` then `$match` on an empty array) finds orphan documents.

Query 31: Weighted rating (IMDb-style Bayesian formula) – Top 10

Purpose: $WR = (v/(v+m)) \cdot R + (m/(v+m)) \cdot C$, where v = votes, m = 50 (minimum), R = movie mean, C = global mean 3.5. Balances quality against popularity.

Query:

```
db.ratings.aggregate([
  {
    $group: { _id: "$movieId", v: { $sum: 1 }, R: { $avg: "$rating" } }
  },
  { $match: { v: { $gte: 50 } } },
  {
    $addFields: {
      weighted: {
        $add: [
          {
            $multiply: [{ $divide: ["$v", { $add: ["$v", 50] } ] }, "$R" ]
          },
          {
            $multiply: [{ $divide: [50, { $add: ["$v", 50] } ] }, 3.5 ]
          }
        ]
      }
    }
  }
])
```

```

    ]
  }
}
},
{ $sort: { weighted: -1 } },
{ $limit: 10 },
{
  $lookup: {
    from: "movies",
    localField: "_id",
    foreignField: "movieId",
    as: "movie"
  }
},
{ $unwind: "$movie" },
{
  $project: {
    _id: 0,
    title: "$movie.title",
    votes: "$v",
    avgRating: { $round: ["$R", 2] },
    weightedRating: { $round: ["$weighted", 3] }
  }
}
])

```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: { _id: "$movieId", v: { $sum: 1 }, R: { $avg: "$rating" } }
...   },
...   { $match: { v: { $gte: 50 } } },
...   {
...     $addFields: {
...       weighted: {
...         $add: [
...           {
...             $multiply: [{ $divide: ["$v", { $add: ["$v", 50] } ] }, "$R" ]
...           },
...           {
...             $multiply: [{ $divide: [50, { $add: ["$v", 50] } ] }, 3.5 ]
...           }
...         ]
...       }
...     }
...   },
...   { $sort: { weighted: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       votes: "$v",
...       avgRating: { $round: ["$R", 2] },
...       weightedRating: { $round: ["$weighted", 3] }
...     }
...   }
... ])

[
  {
    title: 'The Shawshank Redemption (1994)',
    votes: 317,
    avgRating: 4.43,
    weightedRating: 4.302
  },
  {
    title: 'Fight Club (1999)',
    votes: 218,
    avgRating: 4.27,
    weightedRating: 4.129
  },
  {
    title: 'The Godfather (1972)',
    votes: 192,
    avgRating: 4.29,
    weightedRating: 4.126
  },
  {
    title: 'Star Wars: Episode IV - A New Hope (1977)',
    votes: 251,
    avgRating: 4.23,
    weightedRating: 4.11
  },
  {
    title: 'Pulp Fiction (1994)',
    votes: 307,
    avgRating: 4.2,
    weightedRating: 4.099
  },
  {
    title: 'The Usual Suspects (1995)',
    votes: 204,
    avgRating: 4.24,
    weightedRating: 4.093
  },
  {
    title: 'Schindler's List (1993)',
    votes: 220,
    avgRating: 4.22,
    weightedRating: 4.091
  },
  {
    title: 'The Matrix (1999)',
    votes: 278,
    avgRating: 4.19,
    weightedRating: 4.087
  },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    votes: 211,
    avgRating: 4.22,
    weightedRating: 4.079
  },
  {
    title: 'Forrest Gump (1994)',
    votes: 329,
    avgRating: 4.16,
    weightedRating: 4.077
  }
]

cinematch> 
```

Observation: The Bayesian formula shrinks averages of movies with few votes towards 3.5, so the top-10 contains only films that are both popular and highly rated – the basis of the dashboard's recommendations.

11.5 Genre Analysis

Query 32: Average rating per genre

Purpose: Join every movie to its ratings, unwind both arrays, and compute the mean rating and rating volume of each genre.

Query:

```
db.movies.aggregate([
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $unwind: "$r" },
  { $unwind: "$genreList" },
  {
    $group: {
      _id: "$genreList",
      avgRating: { $avg: "$r.rating" },
      ratings: { $sum: 1 }
    }
  },
  {
    $project: { avgRating: { $round: ["$avgRating", 3] }, ratings: 1 }
  },
  { $sort: { avgRating: -1 } }
])
```

Output:

mongosh - cinematch

```
cinematch> db.movies.aggregate([
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   { $unwind: "$r" },
...   { $unwind: "$genreList" },
...   {
...     $group: {
...       _id: "$genreList",
...       avgRating: { $avg: "$r.rating" },
...       ratings: { $sum: 1 }
...     }
...   },
...   {
...     $project: { avgRating: { $round: ["$avgRating", 3] }, ratings: 1 }
...   },
...   { $sort: { avgRating: -1 } }
... ])

[
  { _id: 'Film-Noir', ratings: 870, avgRating: 3.92 },
  { _id: 'War', ratings: 4859, avgRating: 3.808 },
  { _id: 'Documentary', ratings: 1219, avgRating: 3.798 },
  { _id: 'Crime', ratings: 16681, avgRating: 3.658 },
  { _id: 'Drama', ratings: 41928, avgRating: 3.656 },
  { _id: 'Mystery', ratings: 7674, avgRating: 3.632 },
  { _id: 'Animation', ratings: 6988, avgRating: 3.63 },
  { _id: 'IMAX', ratings: 4145, avgRating: 3.618 },
  { _id: 'Western', ratings: 1930, avgRating: 3.584 },
  { _id: 'Musical', ratings: 4138, avgRating: 3.564 },
  { _id: 'Adventure', ratings: 24161, avgRating: 3.509 },
  { _id: 'Romance', ratings: 18124, avgRating: 3.507 },
  { _id: 'Thriller', ratings: 26452, avgRating: 3.494 },
  { _id: 'Fantasy', ratings: 11834, avgRating: 3.491 },
  { _id: '(no genres listed)', ratings: 47, avgRating: 3.489 },
  { _id: 'Sci-Fi', ratings: 17243, avgRating: 3.456 },
  { _id: 'Action', ratings: 30635, avgRating: 3.448 },
  { _id: 'Children', ratings: 9208, avgRating: 3.413 },
  { _id: 'Comedy', ratings: 39053, avgRating: 3.385 },
  { _id: 'Horror', ratings: 7291, avgRating: 3.258 }
]

cinematch> █
```

Observation: Film-Noir (3.92), War (3.81) and Documentary (3.80) receive the highest average ratings; Horror (3.26), Comedy (3.39) and Children (3.41) the lowest – niche genres attract enthusiasts, mass genres attract critics.

Query 33: Most popular genres by number of ratings

Purpose: \$size of the joined ratings array is summed per genre – shows what people actually watch rather than what they rate highly.

Query:

```
db.movies.aggregate([
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  }
])
```

```

    },
    { $project: { genreList: 1, n: { $size: "$n" } } },
    { $unwind: "$genreList" },
    { $group: { _id: "$genreList", ratings: { $sum: "$n" } } },
    { $sort: { ratings: -1 } },
    { $limit: 10 }
  ])
}

```

Output:

```

mongosh - cinematch

cinematch> db.movies.aggregate([
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   { $project: { genreList: 1, n: { $size: "$n" } } },
...   { $unwind: "$genreList" },
...   { $group: { _id: "$genreList", ratings: { $sum: "$n" } } },
...   { $sort: { ratings: -1 } },
...   { $limit: 10 }
... ])

[
  { _id: 'Drama', ratings: 41928 },
  { _id: 'Comedy', ratings: 39053 },
  { _id: 'Action', ratings: 30635 },
  { _id: 'Thriller', ratings: 26452 },
  { _id: 'Adventure', ratings: 24161 },
  { _id: 'Romance', ratings: 18124 },
  { _id: 'Sci-Fi', ratings: 17243 },
  { _id: 'Crime', ratings: 16681 },
  { _id: 'Fantasy', ratings: 11834 },
  { _id: 'Children', ratings: 9208 }
]

cinematch>

```

Observation: By volume, Drama (41,928 ratings) and Comedy (39,053) are the most watched genres, even though they are not the highest rated.

Query 34: Best movie in each genre (minimum 50 ratings)

Purpose: \$sort followed by \$group with \$first picks the top-rated title per genre.

Query:

```

db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {

```

```
{ $unwind: "$movie" },
{ $unwind: "$movie.genreList" },
{ $sort: { avgRating: -1 } },
{
  $group: {
    _id: "$movie.genreList",
    bestMovie: { $first: "$movie.title" },
    avgRating: { $first: { $round: ["$avgRating", 2] } },
    ratingCount: { $first: "$ratingCount" }
  }
},
{ $sort: { _id: 1 } }
])
```

Output:

mongosh - cinematch

— 0 0

```
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     },
...     { $match: { ratingCount: { $gte: 50 } } },
...     {
...       $lookup: {
...         from: "movies",
...         localField: "_id",
...         foreignField: "movieId",
...         as: "movie"
...       }
...     },
...     { $unwind: "$movie" },
...     { $unwind: "$movie.genreList" },
...     { $sort: { avgRating: -1 } },
...     {
...       $group: {
...         _id: "$movie.genreList",
...         bestMovie: { $first: "$movie.title" },
...         avgRating: { $first: { $round: ["$avgRating", 2] } },
...         ratingCount: { $first: "$ratingCount" }
...       }
...     },
...     { $sort: { _id: 1 } }
...   ]
... )
[
  {
    _id: 'Action',
    bestMovie: 'Fight Club (1999)',
    avgRating: 4.27,
    ratingCount: 218
  },
  {
    _id: 'Adventure',
    bestMovie: 'The Princess Bride (1987)',
    avgRating: 4.23,
    ratingCount: 142
  },
  {
    _id: 'Animation',
    bestMovie: 'Spirited Away (Sen to Chihiro no kamikakushi) (2001)',
    avgRating: 4.16,
    ratingCount: 87
  },
  {
    _id: 'Children',
    bestMovie: 'Toy Story 3 (2010)',
    avgRating: 4.11,
    ratingCount: 55
  },
  {
    _id: 'Comedy',
    bestMovie: 'Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1964)',
    avgRating: 4.27,
    ratingCount: 97
  },
  {
    _id: 'Crime',
    bestMovie: 'The Shawshank Redemption (1994)',
    avgRating: 4.43,
    ratingCount: 317
  },
  {
    _id: 'Documentary',
    bestMovie: 'Bowling for Columbine (2002)',
    avgRating: 3.78,
    ratingCount: 58
  },
  {
    _id: 'Drama',
    bestMovie: 'The Shawshank Redemption (1994)',
    avgRating: 4.43,
    ratingCount: 317
  },
  {
    _id: 'Fantasy',
    bestMovie: 'The Princess Bride (1987)',
    avgRating: 4.23,
    ratingCount: 142
  },
  {
    _id: 'Film-Noir',
    bestMovie: 'Chinatown (1974)',
    avgRating: 4.21,
    ratingCount: 59
  },
  {
    _id: 'Horror',
    bestMovie: 'The Silence of the Lambs (1991)',
    ... (52 more lines)
  }
]
cinematch> █
```

Observation: Each genre's best film (min 50 ratings): The Shawshank Redemption for Crime/Drama, Fight Club for Action/Thriller, Spirited Away for Animation, Rear Window for Mystery, Casablanca for Romance and Chinatown for Film-Noir.

Query 35: Genre popularity by decade (Action vs Drama vs Comedy)

Purpose: Two-level grouping (decade + genre) to see how the catalogue mix changed over time.

Query:

```
db.movies.aggregate([
  {
    $match: {
      year: { $gte: 1970 },
      genreList: { $in: ["Action", "Drama", "Comedy"] }
    }
  },
  { $unwind: "$genreList" },
  { $match: { genreList: { $in: ["Action", "Drama", "Comedy"] } } },
  {
    $group: {
      _id: {
        decade: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
        genre: "$genreList"
      },
      movies: { $sum: 1 }
    }
  },
  { $sort: { "_id.decade": 1, "_id.genre": 1 } }
])
```

Output:

mongosh - cinematch

— □ □

```
cinematch> db.movies.aggregate([
...   {
...     $match: {
...       year: { $gte: 1970 },
...       genreList: { $in: ["Action", "Drama", "Comedy"] }
...     }
...   },
...   { $unwind: "$genreList" },
...   { $match: { genreList: { $in: ["Action", "Drama", "Comedy"] } } },
...   {
...     $group: {
...       _id: {
...         decade: { $multiply: [{ $floor: { $divide: ["$year", 10] } }, 10] },
...         genre: "$genreList"
...       },
...       movies: { $sum: 1 }
...     }
...   },
...   { $sort: { "_id.decade": 1, "_id.genre": 1 } }
... ])

[
  { _id: { decade: 1970, genre: 'Action' }, movies: 103 },
  { _id: { decade: 1970, genre: 'Comedy' }, movies: 155 },
  { _id: { decade: 1970, genre: 'Drama' }, movies: 225 },
  { _id: { decade: 1980, genre: 'Action' }, movies: 227 },
  { _id: { decade: 1980, genre: 'Comedy' }, movies: 492 },
  { _id: { decade: 1980, genre: 'Drama' }, movies: 439 },
  { _id: { decade: 1990, genre: 'Action' }, movies: 392 },
  { _id: { decade: 1990, genre: 'Comedy' }, movies: 890 },
  { _id: { decade: 1990, genre: 'Drama' }, movies: 1038 },
  { _id: { decade: 2000, genre: 'Action' }, movies: 549 },
  { _id: { decade: 2000, genre: 'Comedy' }, movies: 1135 },
  { _id: { decade: 2000, genre: 'Drama' }, movies: 1365 },
  { _id: { decade: 2010, genre: 'Action' }, movies: 464 },
  { _id: { decade: 2010, genre: 'Comedy' }, movies: 758 },
  { _id: { decade: 2010, genre: 'Drama' }, movies: 743 }
]

cinematch> █
```

Observation: Drama out-produced Comedy and Action in every decade; all three peaked in the 2000s, reflecting catalogue growth rather than a change in taste.

Query 36: Average number of genres per movie

Purpose: \$size on the genreList array, averaged across the catalogue.

Query:

```
db.movies.aggregate([
  { $project: { genreCount: { $size: "$genreList" } } },
  {
    $group: {
      _id: null,
      avgGenres: { $avg: "$genreCount" },
      maxGenres: { $max: "$genreCount" }
    }
  },
  {
    $project: {
      _id: 0,
      avgGenres: { $round: ["$avgGenres", 2] },
      maxGenres: 1
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $project: { genreCount: { $size: "$genreList" } } },
...   {
...     $group: {
...       _id: null,
...       avgGenres: { $avg: "$genreCount" },
...       maxGenres: { $max: "$genreCount" }
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       avgGenres: { $round: ["$avgGenres", 2] },
...       maxGenres: 1
...     }
...   }
... ])

[ { maxGenres: 10, avgGenres: 2.27 } ]

cinematch>
```

Observation: Movies carry on average 2.27 genres; the maximum is 10 (a single title tagged with almost every genre).

11.6 User Analysis

Query 37: Top 10 most active users

Purpose: Users ranked by number of ratings submitted, with each user's average score.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $sort: { ratings: -1 } },
  { $limit: 10 },
  {
    $project: {
      userId: "$_id",
      _id: 0,
      ratings: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$userId",
...       ratings: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   { $sort: { ratings: -1 } },
...   { $limit: 10 },
...   {
...     $project: {
...       userId: "$_id",
...       _id: 0,
...       ratings: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])
[
  { ratings: 2698, userId: 414, avgRating: 3.39 },
  { ratings: 2478, userId: 599, avgRating: 2.64 },
  { ratings: 2108, userId: 474, avgRating: 3.4 },
  { ratings: 1864, userId: 448, avgRating: 2.85 },
  { ratings: 1346, userId: 274, avgRating: 3.24 },
  { ratings: 1302, userId: 610, avgRating: 3.69 },
  { ratings: 1260, userId: 68, avgRating: 3.23 },
  { ratings: 1218, userId: 380, avgRating: 3.67 },
  { ratings: 1115, userId: 606, avgRating: 3.66 },
  { ratings: 1055, userId: 288, avgRating: 3.15 }
]

cinematch>
```

Observation: User 414 rated 2,698 movies, followed by users 599 (2,478) and 474 (2,108) – a handful of super-users contribute a large share of all ratings.

Query 38: Harshest and most generous critics (minimum 100 ratings)

Purpose: \$facet runs two sub-pipelines in one query: lowest and highest average scorers.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$userId",
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  { $match: { ratings: { $gte: 100 } } },
  {
    $project: {
      userId: "$_id",
      _id: 0,
      ratings: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  },
  {
    $facet: {
      harshest: [{ $sort: { avgRating: 1 } }, { $limit: 5 }],
      mostGenerous: [{ $sort: { avgRating: -1 } }, { $limit: 5 } ]
    }
  }
])
```

```
}  
])
```

Output:

mongosh - cinematch

```
cinematch> db.ratings.aggregate([  
...   {  
...     $group: {  
...       _id: "$userId",  
...       ratings: { $sum: 1 },  
...       avgRating: { $avg: "$rating" }  
...     }  
...   },  
...   { $match: { ratings: { $gte: 100 } } },  
...   {  
...     $project: {  
...       userId: "$_id",  
...       _id: 0,  
...       ratings: 1,  
...       avgRating: { $round: ["$avgRating", 2] }  
...     }  
...   },  
...   {  
...     $facet: {  
...       harshest: [{ $sort: { avgRating: 1 } }, { $limit: 5 }],  
...       mostGenerous: [{ $sort: { avgRating: -1 } }, { $limit: 5 }]  
...     }  
...   }  
... ])  
  
[  
  {  
    harshest: [  
      { ratings: 194, userId: 139, avgRating: 2.14 },  
      { ratings: 179, userId: 153, avgRating: 2.22 },  
      { ratings: 385, userId: 567, avgRating: 2.25 },  
      { ratings: 939, userId: 298, avgRating: 2.36 },  
      { ratings: 400, userId: 517, avgRating: 2.39 }  
    ],  
    mostGenerous: [  
      { ratings: 202, userId: 452, avgRating: 4.56 },  
      { ratings: 292, userId: 122, avgRating: 4.55 },  
      { ratings: 114, userId: 43, avgRating: 4.55 },  
      { ratings: 130, userId: 52, avgRating: 4.48 },  
      { ratings: 101, userId: 601, avgRating: 4.43 }  
    ]  
  }  
]  
  
cinematch> 
```

Observation: \$facet returns both lists in one query: user 139 averages only 2.14 stars across 194 ratings while user 452 averages 4.56 – rating scales are highly subjective, which is why per-user normalisation matters in recommenders.

Query 39: Distribution of ratings per user

Purpose: \$bucket groups users into activity bands (20–49, 50–99, ... ratings).

Query:

```
db.ratings.aggregate([  
  { $group: { _id: "$userId", ratings: { $sum: 1 } } },  
  {  
    $bucket: {  
      groupBy: "$ratings",  
      boundaries: [20, 50, 100, 200, 500, 1000, 3000],  
      default: "3000+",  
      output: { users: { $sum: 1 } }  
    }  
  }  
])
```

```
}  
])
```

Output:

```
mongosh - cinematch  
  
cinematch> db.ratings.aggregate([  
...   { $group: { _id: "$userId", ratings: { $sum: 1 } } },  
...   {  
...     $bucket: {  
...       groupBy: "$ratings",  
...       boundaries: [20, 50, 100, 200, 500, 1000, 3000],  
...       default: "3000+",  
...       output: { users: { $sum: 1 } }  
...     }  
...   }  
... ])  
  
[  
  { _id: 20, users: 225 },  
  { _id: 50, users: 137 },  
  { _id: 100, users: 114 },  
  { _id: 200, users: 90 },  
  { _id: 500, users: 32 },  
  { _id: 1000, users: 12 }  
]  
  
cinematch> █
```

Observation: 225 users have 20–49 ratings and 137 have 50–99; only 12 users exceed 1,000 ratings – a classic long-tail activity distribution.

Query 40: Favourite genres of a specific user (userId 1)

Purpose: Personal taste profile: which genres does user 1 rate most often and most highly?

Query:

```
db.ratings.aggregate([  
  { $match: { userId: 1 } },  
  {  
    $lookup: {  
      from: "movies",  
      localField: "movieId",  
      foreignField: "movieId",  
      as: "movie"  
    }  
  },  
  { $unwind: "$movie" },  
  { $unwind: "$movie.genreList" },  
  {  
    $group: {  
      _id: "$movie.genreList",  
      rated: { $sum: 1 },  
      avgRating: { $avg: "$rating" }  
    }  
  },  
  {  
    $project: { rated: 1, avgRating: { $round: ["$avgRating", 2] } }  
  },  
  { $sort: { rated: -1 } },  
  { $limit: 5 }  
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { userId: 1 } },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   { $unwind: "$movie.genreList" },
...   {
...     $group: {
...       _id: "$movie.genreList",
...       rated: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   {
...     $project: { rated: 1, avgRating: { $round: ["$avgRating", 2] } }
...   },
...   { $sort: { rated: -1 } },
...   { $limit: 5 }
... ])

[
  { _id: 'Action', rated: 90, avgRating: 4.32 },
  { _id: 'Adventure', rated: 85, avgRating: 4.39 },
  { _id: 'Comedy', rated: 83, avgRating: 4.28 },
  { _id: 'Drama', rated: 68, avgRating: 4.53 },
  { _id: 'Thriller', rated: 55, avgRating: 4.15 }
]

cinematch> 
```

Observation: User 1's most-rated genres are Action, Adventure and Comedy, with Drama rated highest (4.53) – this profile drives the personalised recommendations in Query 51.

Query 41: Number of ratings submitted per year

Purpose: \$year on the ratedAt date field shows activity on the platform from 1996 to 2018 (.toArray() prints all 23 years instead of the first 20 cursor results).

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: { $year: "$ratedAt" },
      ratings: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  {
    $project: { ratings: 1, avgRating: { $round: ["$avgRating", 2] } }
  },
  { $sort: { _id: 1 } }
]).toArray()
```

Output:

mongosh - cinematch

```
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: { $year: "$ratedAt" },
...       ratings: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   {
...     $project: { ratings: 1, avgRating: { $round: ["$avgRating", 2] } }
...   },
...   { $sort: { _id: 1 } }
... ]).toArray()

[
  { _id: 1996, ratings: 6040, avgRating: 3.54 },
  { _id: 1997, ratings: 1916, avgRating: 3.73 },
  { _id: 1998, ratings: 507, avgRating: 3.44 },
  { _id: 1999, ratings: 2439, avgRating: 3.77 },
  { _id: 2000, ratings: 10061, avgRating: 3.39 },
  { _id: 2001, ratings: 3922, avgRating: 3.51 },
  { _id: 2002, ratings: 3478, avgRating: 3.61 },
  { _id: 2003, ratings: 4014, avgRating: 3.5 },
  { _id: 2004, ratings: 3279, avgRating: 3.51 },
  { _id: 2005, ratings: 5813, avgRating: 3.43 },
  { _id: 2006, ratings: 4059, avgRating: 3.49 },
  { _id: 2007, ratings: 7114, avgRating: 3.31 },
  { _id: 2008, ratings: 4351, avgRating: 3.66 },
  { _id: 2009, ratings: 4158, avgRating: 3.39 },
  { _id: 2010, ratings: 2301, avgRating: 3.46 },
  { _id: 2011, ratings: 1690, avgRating: 3.78 },
  { _id: 2012, ratings: 4656, avgRating: 3.64 },
  { _id: 2013, ratings: 1664, avgRating: 3.88 },
  { _id: 2014, ratings: 1439, avgRating: 3.51 },
  { _id: 2015, ratings: 6616, avgRating: 3.51 },
  { _id: 2016, ratings: 6703, avgRating: 3.69 },
  { _id: 2017, ratings: 8198, avgRating: 3.35 },
  { _id: 2018, ratings: 6418, avgRating: 3.39 }
]

cinematch>
```

Observation: Rating activity peaks in 2000 (10,061 ratings), 2007 (7,114) and 2015 (6,616); the average rating per year stays within 3.3–3.9 despite large swings in volume.

Query 42: Ratings by day of the week

Purpose: `$dayOfWeek` (1 = Sunday ... 7 = Saturday) reveals when users are most active.

Query:

```
db.ratings.aggregate([
  {
    $group: { _id: { $dayOfWeek: "$ratedAt" }, ratings: { $sum: 1 } }
  },
  { $sort: { _id: 1 } },
  {
    $project: {
      _id: 0,
      day: {
        $arrayElemAt: [
          [ "", "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"],
          "$_id"
        ]
      },
      ratings: 1
    }
  }
])
```

```
}  
])
```

Output:

```
mongosh - cinematch  
  
cinematch> db.ratings.aggregate([  
...   {  
...     $group: { _id: { $dayOfWeek: "$ratedAt" }, ratings: { $sum: 1 } }  
...   },  
...   { $sort: { _id: 1 } },  
...   {  
...     $project: {  
...       _id: 0,  
...       day: {  
...         $arrayElemAt: [  
...           [ "", "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"],  
...           "$_id"  
...         ]  
...       },  
...       ratings: 1  
...     }  
...   }  
... ])  
  
[  
  { ratings: 16209, day: 'Sun' },  
  { ratings: 17583, day: 'Mon' },  
  { ratings: 16411, day: 'Tue' },  
  { ratings: 14014, day: 'Wed' },  
  { ratings: 10706, day: 'Thu' },  
  { ratings: 14455, day: 'Fri' },  
  { ratings: 11458, day: 'Sat' }  
]  
  
cinematch> █
```

Observation: Monday (17,583) is the busiest day and Thursday (10,706) the quietest; activity is otherwise spread fairly evenly across the week.

Query 43: First and last rating date of the most active user

Purpose: \$min/\$max on dates give the active lifespan of user 414.

Query:

```
db.ratings.aggregate([  
  { $match: { userId: 414 } },  
  {  
    $group: {  
      _id: "$userId",  
      firstRating: { $min: "$ratedAt" },  
      lastRating: { $max: "$ratedAt" },  
      total: { $sum: 1 }  
    }  
  }  
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { userId: 414 } },
...   {
...     $group: {
...       _id: "$userId",
...       firstRating: { $min: "$ratedAt" },
...       lastRating: { $max: "$ratedAt" },
...       total: { $sum: 1 }
...     }
...   }
... ])

[
  {
    _id: 414,
    firstRating: ISODate('2000-06-19T17:36:56.000Z'),
    lastRating: ISODate('2018-06-02T22:21:12.000Z'),
    total: 2698
  }
]

cinematch>
```

Observation: User 414 was active from June 2000 to June 2018 – an 18-year rating history.

11.7 Tag Analysis

Query 44: Most frequently used tags

Purpose: `$toLower` normalises case before counting so 'Funny' and 'funny' merge.

Query:

```
db.tags.aggregate([
  { $group: { _id: { $toLower: "$tag" }, count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 15 }
])
```

Output:

```
mongosh - cinematch

cinematch> db.tags.aggregate([
...   { $group: { _id: { $toLower: "$tag" }, count: { $sum: 1 } } },
...   { $sort: { count: -1 } },
...   { $limit: 15 }
... ])

[
  { _id: 'in netflix queue', count: 131 },
  { _id: 'atmospheric', count: 41 },
  { _id: 'surreal', count: 24 },
  { _id: 'thought-provoking', count: 24 },
  { _id: 'funny', count: 24 },
  { _id: 'superhero', count: 24 },
  { _id: 'disney', count: 23 },
  { _id: 'sci-fi', count: 23 },
  { _id: 'religion', count: 22 },
  { _id: 'quirky', count: 22 },
  { _id: 'psychology', count: 21 },
  { _id: 'dark comedy', count: 21 },
  { _id: 'suspense', count: 21 },
  { _id: 'visually appealing', count: 20 },
  { _id: 'twist ending', count: 20 }
]

cinematch>
```

Observation: 'in netflix queue' (131) is the most common tag, followed by 'atmospheric', 'thought-provoking' and 'superhero'; \$toLower merges case variants.

Query 45: All tags applied to Pulp Fiction

Purpose: \$addToSet collects the unique tags for movieId 296; \$slice shows the first 15 of them.

Query:

```
db.tags.aggregate([
  { $match: { movieId: 296 } },
  {
    $group: {
      _id: "$movieId",
      tags: { $addToSet: "$tag" },
      taggers: { $addToSet: "$userId" }
    }
  },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {
    $project: {
      _id: 0,
      title: { $first: "$movie.title" },
      totalTags: { $size: "$tags" },
      taggerCount: { $size: "$taggers" },
      sampleTags: { $slice: ["$tags", 15] }
    }
  }
])
```

Output:

mongosh - cinematch

```
cinematch> db.tags.aggregate([
...   { $match: { movieId: 296 } },
...   {
...     $group: {
...       _id: "$movieId",
...       tags: { $addToSet: "$tag" },
...       taggers: { $addToSet: "$userId" }
...     }
...   },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       title: { $first: "$movie.title" },
...       totalTags: { $size: "$tags" },
...       taggerCount: { $size: "$taggers" },
...       sampleTags: { $slice: ["$tags", 15] }
...     }
...   }
... ])
[
  {
    title: 'Pulp Fiction (1994)',
    totalTags: 173,
    taggerCount: 4,
    sampleTags: [
      'Palme d'Or',
      'r:strong bloody violence',
      'stylish',
      'mobster',
      'characters',
      'drugs',
      'great acting',
      'rape',
      'film noir',
      'r:graphic sexuality',
      'masterpiece',
      'coke',
      'nonlinear',
      'dark humor',
      'exciting'
    ]
  }
]
cinematch> 
```

Observation: Pulp Fiction carries 173 distinct tags from just 4 users – tagging is concentrated among a few enthusiastic users.

Query 46: Most tagged movies

Purpose: Which films attract the most free-text annotation from users?

Query:

```
db.tags.aggregate([
  { $group: { _id: "$movieId", tagCount: { $sum: 1 } } },
  { $sort: { tagCount: -1 } },
  { $limit: 10 },
  {
    $lookup: {
```

```

        from: "movies",
        localField: "_id",
        foreignField: "movieId",
        as: "movie"
    }
},
{ $unwind: "$movie" },
{ $project: { _id: 0, title: "$movie.title", tagCount: 1 } }
])

```

Output:

```

mongosh - cinematch

cinematch> db.tags.aggregate([
...   { $group: { _id: "$movieId", tagCount: { $sum: 1 } } },
...   { $sort: { tagCount: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   { $project: { _id: 0, title: "$movie.title", tagCount: 1 } }
... ])

[
  { tagCount: 181, title: 'Pulp Fiction (1994)' },
  { tagCount: 54, title: 'Fight Club (1999)' },
  { tagCount: 41, title: '2001: A Space Odyssey (1968)' },
  {
    tagCount: 35,
    title: 'Léon: The Professional (a.k.a. The Professional) (Léon) (1994)'
  },
  {
    tagCount: 34,
    title: 'Eternal Sunshine of the Spotless Mind (2004)'
  },
  { tagCount: 32, title: 'The Big Lebowski (1998)' },
  { tagCount: 29, title: 'Donnie Darko (2001)' },
  { tagCount: 26, title: 'Inception (2010)' },
  { tagCount: 26, title: 'Star Wars: Episode IV - A New Hope (1977)' },
  { tagCount: 19, title: 'Suicide Squad (2016)' }
]

cinematch>

```

Observation: Pulp Fiction (181 tags) and Fight Club (54) are the most tagged movies – cult classics attract the most annotation.

Query 47: Average rating of movies tagged 'funny'

Purpose: Combines tags and ratings through two \$lookup stages to test whether comedies tagged funny are rated well.

Query:

```

db.tags.aggregate([
  { $match: { tag: { $regex: "^funny$", $options: "i" } } },
  { $group: { _id: "$movieId" } },
  {
    $lookup: {
      from: "ratings",
      localField: "_id",
      foreignField: "movieId",
      as: "r"
    }
  }
])

```

```

    }
  },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {
    $project: {
      _id: 0,
      title: { $first: "$movie.title" },
      ratings: { $size: "$r" },
      avgRating: { $round: [{ $avg: "$r.rating" }, 2] }
    }
  },
  { $sort: { ratings: -1 } },
  { $limit: 10 }
])

```

Output:

mongosh - cinematch

```
cinematch> db.tags.aggregate([
...   { $match: { tag: { $regex: "^funny$", $options: "i" } } },
...   { $group: { _id: "$movieId" } },
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   {
...     $project: {
...       _id: 0,
...       title: { $first: "$movie.title" },
...       ratings: { $size: "$r" },
...       avgRating: { $round: [{ $avg: "$r.rating" }, 2] }
...     }
...   },
...   { $sort: { ratings: -1 } },
...   { $limit: 10 }
... ])

[
  { title: 'Pulp Fiction (1994)', ratings: 307, avgRating: 4.2 },
  { title: 'The Big Lebowski (1998)', ratings: 106, avgRating: 3.92 },
  { title: 'Clueless (1995)', ratings: 104, avgRating: 3.29 },
  { title: 'Toy Story 2 (1999)', ratings: 97, avgRating: 3.86 },
  { title: 'The Hangover (2009)', ratings: 76, avgRating: 3.63 },
  { title: 'Django Unchained (2012)', ratings: 71, avgRating: 3.94 },
  {
    title: 'Guardians of the Galaxy (2014)',
    ratings: 59,
    avgRating: 4.05
  },
  { title: 'Zombieland (2009)', ratings: 53, avgRating: 3.88 },
  {
    title: '(500) Days of Summer (2009)',
    ratings: 42,
    avgRating: 3.67
  },
  {
    title: 'Home Alone 2: Lost in New York (1992)',
    ratings: 31,
    avgRating: 2.52
  }
]
```

Observation: Films tagged 'funny' range from Pulp Fiction (4.20) and Guardians of the Galaxy (4.05) down to Home Alone 2 (2.52) – the double \$lookup combines all three collections in one pipeline.

Query 48: Most active taggers

Purpose: Users grouped by number of tag applications; a handful of users generate most tags.

Query:

```
db.tags.aggregate([
  {
    $group: {
      _id: "$userId",
```

```

    tags: { $sum: 1 },
    movies: { $addToSet: "$movieId" }
  }
},
{
  $project: {
    userId: "$_id",
    _id: 0,
    tags: 1,
    moviesTagged: { $size: "$movies" }
  }
},
{ $sort: { tags: -1 } },
{ $limit: 5 }
])

```

Output:



```

mongosh - cinematch

cinematch> db.tags.aggregate([
...   {
...     $group: {
...       _id: "$userId",
...       tags: { $sum: 1 },
...       movies: { $addToSet: "$movieId" }
...     }
...   },
...   {
...     $project: {
...       userId: "$_id",
...       _id: 0,
...       tags: 1,
...       moviesTagged: { $size: "$movies" }
...     }
...   },
...   { $sort: { tags: -1 } },
...   { $limit: 5 }
... ])

[
  { tags: 1507, userId: 474, moviesTagged: 1235 },
  { tags: 432, userId: 567, moviesTagged: 109 },
  { tags: 370, userId: 62, moviesTagged: 69 },
  { tags: 323, userId: 599, moviesTagged: 5 },
  { tags: 280, userId: 477, moviesTagged: 68 }
]

cinematch>

```

Observation: User 474 applied 1,507 of the 3,683 tags (41 %) – a single user dominates the tags collection.

11.8 Recommendation Queries

Query 49: Content-based: movies most similar to Toy Story by genre overlap

Purpose: \$setIntersection counts shared genres with Toy Story's genreList; ties are broken by average rating (min 20 ratings).

Query:

```

db.movies.aggregate([
  { $match: { movieId: { $ne: 1 } } },
  {
    $addFields: {
      sharedGenres: {
        $size: {

```

```

        $setIntersection: [
            "$genreList",
            [
                "Adventure",
                "Animation",
                "Children",
                "Comedy",
                "Fantasy"
            ]
        ]
    }
}
},
{ $match: { sharedGenres: { $gte: 4 } } },
{
    $lookup: {
        from: "ratings",
        localField: "movieId",
        foreignField: "movieId",
        as: "r"
    }
},
{
    $addFields: {
        ratingCount: { $size: "$r" },
        avgRating: { $avg: "$r.rating" }
    }
},
{ $match: { ratingCount: { $gte: 20 } } },
{ $sort: { sharedGenres: -1, avgRating: -1 } },
{ $limit: 10 },
{
    $project: {
        _id: 0,
        title: 1,
        sharedGenres: 1,
        ratingCount: 1,
        avgRating: { $round: ["$avgRating", 2] }
    }
}
])

```

Output:

```
mongosh - cinematch

cinematch> db.movies.aggregate([
...   { $match: { movieId: { $ne: 1 } } },
...   {
...     $addFields: {
...       sharedGenres: {
...         $size: {
...           $setIntersection: [
...             "$genreList",
...             [
...               "Adventure",
...               "Animation",
...               "Children",
...               "Comedy",
...               "Fantasy"
...             ]
...           }
...         }
...       }
...     },
...     { $match: { sharedGenres: { $gte: 4 } } },
...     {
...       $lookup: {
...         from: "ratings",
...         localField: "movieId",
...         foreignField: "movieId",
...         as: "r"
...       }
...     },
...     {
...       $addFields: {
...         ratingCount: { $size: "$r" },
...         avgRating: { $avg: "$r.rating" }
...       }
...     },
...     { $match: { ratingCount: { $gte: 20 } } },
...     { $sort: { sharedGenres: -1, avgRating: -1 } },
...     { $limit: 10 },
...     {
...       $project: {
...         _id: 0,
...         title: 1,
...         sharedGenres: 1,
...         ratingCount: 1,
...         avgRating: { $round: ["$avgRating", 2] }
...       }
...     }
...   })
... ]
[
  {
    title: 'Toy Story 3 (2010)',
    sharedGenres: 5,
    ratingCount: 55,
    avgRating: 4.11
  },
  {
    title: 'Monsters, Inc. (2001)',
    sharedGenres: 5,
    ratingCount: 132,
    avgRating: 3.87
  },
  {
    title: 'The Lego Movie (2014)',
    sharedGenres: 5,
    ratingCount: 31,
    avgRating: 3.87
  },
  {
    title: 'Shrek (2001)',
    sharedGenres: 5,
    ratingCount: 170,
    avgRating: 3.87
  },
  {
    title: 'Toy Story 2 (1999)',
    sharedGenres: 5,
    ratingCount: 97,
    avgRating: 3.86
  },
  {
    title: 'Inside Out (2015)',
    sharedGenres: 5,
    ratingCount: 43,
    avgRating: 3.81
  },
  {
    title: 'The Emperor's New Groove (2000)',
    sharedGenres: 5,
    ratingCount: 37,
    avgRating: 3.72
  },
  {
    title: 'Who Framed Roger Rabbit? (1988)',
    sharedGenres: 5,
    ratingCount: 97,
    avgRating: 3.57
  },
  {
    title: 'Antz (1998)',
    sharedGenres: 5,
    ratingCount: 45,
    avgRating: 3.24
  },
  {
    title: 'Shrek the Third (2007)',
    sharedGenres: 5,
    ratingCount: 21,
    avgRating: 3.02
  }
]

cinematch> █
```

Observation: Movies sharing all five of Toy Story's genres are Toy Story 3, Monsters Inc., The Lego Movie, Shrek and Toy Story 2 – content-based similarity finds its sequels and peers purely from metadata.

Query 50: Collaborative: 'users who liked The Matrix also liked...'

Purpose: Find users who gave The Matrix (movieId 2571) ≥ 4.5 , then rank the other movies those users also rated ≥ 4.5 .

Query:

```
db.ratings.aggregate([
  { $match: { movieId: 2571, rating: { $gte: 4.5 } } },
  { $group: { _id: null, fans: { $addToSet: "$userId" } } },
  {
    $lookup: {
      from: "ratings",
      let: { fans: "$fans" },
      pipeline: [
        {
          $match: {
            $expr: {
              $and: [
                { $in: ["$userId", "$$fans"] },
                { $gte: ["$rating", 4.5] },
                { $ne: ["$movieId", 2571] }
              ]
            }
          }
        },
        { $group: { _id: "$movieId", fanCount: { $sum: 1 } } },
        { $sort: { fanCount: -1 } },
        { $limit: 10 }
      ],
      as: "alsoLiked"
    }
  },
  { $unwind: "$alsoLiked" },
  {
    $lookup: {
      from: "movies",
      localField: "alsoLiked._id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {
    $project: {
      _id: 0,
      title: { $first: "$movie.title" },
      fansWhoLikedIt: "$alsoLiked.fanCount",
      totalMatrixFans: { $size: "$fans" }
    }
  }
])
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   { $match: { movieId: 2571, rating: { $gte: 4.5 } } },
...   { $group: { _id: null, fans: { $addToSet: "$userId" } } },
...   {
...     $lookup: {
...       from: "ratings",
...       let: { fans: "$fans" },
...       pipeline: [
...         {
...           $match: {
...             $expr: {
...               $and: [
...                 { $in: [ "$userId", "$$fans" ] },
...                 { $gte: [ "$rating", 4.5 ] },
...                 { $ne: [ "$movieId", 2571 ] }
...               ]
...             }
...           }
...         },
...         { $group: { _id: "$movieId", fanCount: { $sum: 1 } } },
...         { $sort: { fanCount: -1 } },
...         { $limit: 10 }
...       ],
...       as: "alsoLiked"
...     },
...     { $unwind: "$alsoLiked" },
...     {
...       $lookup: {
...         from: "movies",
...         localField: "alsoLiked._id",
...         foreignField: "movieId",
...         as: "movie"
...       }
...     },
...     {
...       $project: {
...         _id: 0,
...         title: { $first: "$movie.title" },
...         fansWhoLikedIt: "$alsoLiked.fanCount",
...         totalMatrixFans: { $size: "$fans" }
...       }
...     }
...   ]
... })

[
  {
    title: 'Fight Club (1999)',
    fansWhoLikedIt: 75,
    totalMatrixFans: 150
  },
  {
    title: 'The Shawshank Redemption (1994)',
    fansWhoLikedIt: 75,
    totalMatrixFans: 150
  },
  {
    title: 'Pulp Fiction (1994)',
    fansWhoLikedIt: 61,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode IV - A New Hope (1977)',
    fansWhoLikedIt: 60,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode V - The Empire Strikes Back (1980)',
    fansWhoLikedIt: 59,
    totalMatrixFans: 150
  },
  {
    title: 'Forrest Gump (1994)',
    fansWhoLikedIt: 57,
    totalMatrixFans: 150
  },
  {
    title: 'The Lord of the Rings: The Fellowship of the Ring (2001)',
    fansWhoLikedIt: 56,
    totalMatrixFans: 150
  },
  {
    title: 'The Silence of the Lambs (1991)',
    fansWhoLikedIt: 51,
    totalMatrixFans: 150
  },
  {
    title: 'Star Wars: Episode VI - Return of the Jedi (1983)',
    fansWhoLikedIt: 50,
    totalMatrixFans: 150
  },
  {
    title: 'Saving Private Ryan (1998)',
    fansWhoLikedIt: 49,
    totalMatrixFans: 150
  }
]

cinematch> 
```

Observation: Of 150 users who rated The Matrix 4.5 or 5, half also loved The Shawshank Redemption and Fight Club, and 60 loved Star Wars – a simple but effective item-to-item collaborative recommendation.

Query 51: Personalised recommendations for user 1

Purpose: Top weighted-rated Action/Adventure movies (user 1's favourite genres) that user 1 has NOT rated yet.

Query:

```
db.movies.aggregate([
  { $match: { genreList: { $all: ["Action", "Adventure"] } } },
  {
    $lookup: {
      from: "ratings",
      localField: "movieId",
      foreignField: "movieId",
      as: "r"
    }
  },
  { $match: { "r.userId": { $ne: 1 } } },
  {
    $addFields: {
      ratingCount: { $size: "$r" },
      avgRating: { $avg: "$r.rating" }
    }
  },
  { $match: { ratingCount: { $gte: 50 } } },
  { $sort: { avgRating: -1 } },
  { $limit: 10 },
  {
    $project: {
      _id: 0,
      title: 1,
      year: 1,
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

mongosh - cinematch

```
cinematch> db.movies.aggregate([
...   { $match: { genreList: { $all: ["Action", "Adventure"] } } },
...   {
...     $lookup: {
...       from: "ratings",
...       localField: "movieId",
...       foreignField: "movieId",
...       as: "r"
...     }
...   },
...   { $match: { "r.userId": { $ne: 1 } } },
...   {
...     $addFields: {
...       ratingCount: { $size: "$r" },
...       avgRating: { $avg: "$r.rating" }
...     }
...   },
...   { $match: { ratingCount: { $gte: 50 } } },
...   { $sort: { avgRating: -1 } },
...   { $limit: 10 },
...   {
...     $project: {
...       _id: 0,
...       title: 1,
...       year: 1,
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])

[
  {
    title: 'North by Northwest (1959)',
    year: 1959,
    ratingCount: 57,
    avgRating: 4.18
  },
  {
    title: 'City of God (Cidade de Deus) (2002)',
    year: 2002,
    ratingCount: 75,
    avgRating: 4.15
  },
  {
    title: 'The Good, the Bad and the Ugly (Buono, il brutto, il cattivo, Il) (1966)',
    year: 1966,
    ratingCount: 72,
    avgRating: 4.15
  },
  {
    title: 'The Lord of the Rings: The Return of the King (2003)',
    year: 2003,
    ratingCount: 185,
    avgRating: 4.12
  },
  {
    title: 'Guardians of the Galaxy (2014)',
    year: 2014,
    ratingCount: 59,
    avgRating: 4.05
  },
  {
    title: 'Army of Darkness (1993)',
    year: 1993,
    ratingCount: 51,
    avgRating: 4.04
  },
  {
    title: 'The Dark Knight Rises (2012)',
    year: 2012,
    ratingCount: 76,
    avgRating: 3.99
  },
  {
    title: 'Aliens (1986)',
    year: 1986,
    ratingCount: 126,
    avgRating: 3.96
  },
  {
    title: 'Children of Men (2006)',
    year: 2006,
    ratingCount: 64,
    avgRating: 3.95
  },
  {
    title: 'Casino Royale (2006)',
    year: 2006,
    ratingCount: 81,
    avgRating: 3.94
  }
]

cinematch> 
```

Observation: The pipeline recommends highly-rated Action/Adventure films that user 1 has not rated (anti-join on r.userId): North by Northwest, City of God, The Good, the Bad and the Ugly and The Return of the King.

Query 52: Hidden gems: highly rated but rarely watched

Purpose: Movies averaging ≥ 4.3 with between 10 and 30 ratings – quality titles that never reached a mass audience.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" }
    }
  },
  {
    $match: { ratingCount: { $gte: 10, $lte: 30 }, avgRating: { $gte: 4.3 } }
  },
  { $sort: { avgRating: -1 } },
  { $limit: 10 },
  {
    $lookup: {
      from: "movies",
      localField: "_id",
      foreignField: "movieId",
      as: "movie"
    }
  },
  {
    $unwind: "$movie"
  },
  {
    $project: {
      _id: 0,
      title: "$movie.title",
      genres: "$movie.genres",
      ratingCount: 1,
      avgRating: { $round: ["$avgRating", 2] }
    }
  }
])
```

Output:

mongosh - cinematch

```
cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" }
...     }
...   },
...   {
...     $match: { ratingCount: { $gte: 10, $lte: 30 }, avgRating: { $gte: 4.3 } }
...   },
...   { $sort: { avgRating: -1 } },
...   { $limit: 10 },
...   {
...     $lookup: {
...       from: "movies",
...       localField: "_id",
...       foreignField: "movieId",
...       as: "movie"
...     }
...   },
...   { $unwind: "$movie" },
...   {
...     $project: {
...       _id: 0,
...       title: "$movie.title",
...       genres: "$movie.genres",
...       ratingCount: 1,
...       avgRating: { $round: ["$avgRating", 2] }
...     }
...   }
... ])
```

```
[
  {
    ratingCount: 11,
    title: 'Secrets & Lies (1996)',
    genres: 'Drama',
    avgRating: 4.59
  },
  {
    ratingCount: 11,
    title: 'Guess Who's Coming to Dinner (1967)',
    genres: 'Drama',
    avgRating: 4.55
  },
  {
    ratingCount: 12,
    title: 'Paths of Glory (1957)',
    genres: 'Drama|War',
    avgRating: 4.54
  },
  {
    ratingCount: 20,
    title: 'A Streetcar Named Desire (1951)',
    genres: 'Drama',
    avgRating: 4.47
  },
  {
    ratingCount: 12,
    title: 'The Celebration (Festen) (1998)',
    genres: 'Drama',
    avgRating: 4.46
  },
  {
    ratingCount: 15,
    title: 'Ran (1985)',
    genres: 'Drama|War',
    avgRating: 4.43
  },
  {
    ratingCount: 14,
    title: 'His Girl Friday (1940)',
    genres: 'Comedy|Romance',
    avgRating: 4.39
  },
  {
    ratingCount: 10,
    title: 'All Quiet on the Western Front (1930)',
    genres: 'Action|Drama|War',
    avgRating: 4.35
  },
  {
    ratingCount: 27,
    title: 'Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)',
    genres: 'Drama|Film-Noir|Romance',
    avgRating: 4.33
  },
  {
    ratingCount: 18,
    title: 'The Hustler (1961)',
    genres: 'Drama',
    avgRating: 4.33
  }
]
```

cinematch> █

Observation: Hidden gems such as Secrets & Lies (4.59 from 11 ratings), Paths of Glory and A Streetcar Named Desire average ≥ 4.3 with only 10–30 ratings – quality titles that a popularity-based list would never surface.

Query 53: Materialise per-movie statistics with \$merge

Purpose: Pre-computes rating count, average and std-dev for every movie into a movie_stats collection so the dashboard can read it without re-aggregating 100k ratings.

Query:

```
db.ratings.aggregate([
  {
    $group: {
      _id: "$movieId",
      ratingCount: { $sum: 1 },
      avgRating: { $avg: "$rating" },
      stdDev: { $stdDevPop: "$rating" }
    }
  },
  { $merge: { into: "movie_stats", whenMatched: "replace" } }
]);
db.movie_stats.find().sort({ ratingCount: -1 }).limit(5)
```

Output:

```
mongosh - cinematch

cinematch> db.ratings.aggregate([
...   {
...     $group: {
...       _id: "$movieId",
...       ratingCount: { $sum: 1 },
...       avgRating: { $avg: "$rating" },
...       stdDev: { $stdDevPop: "$rating" }
...     }
...   },
...   { $merge: { into: "movie_stats", whenMatched: "replace" } }
... ]);
... db.movie_stats.find().sort({ ratingCount: -1 }).limit(5)

[
  {
    _id: 356,
    ratingCount: 329,
    avgRating: 4.164133738601824,
    stdDev: 0.829979269004478
  },
  {
    _id: 318,
    ratingCount: 317,
    avgRating: 4.429022082018927,
    stdDev: 0.7118931016352933
  },
  {
    _id: 296,
    ratingCount: 307,
    avgRating: 4.197068403908795,
    stdDev: 0.9504453980131069
  },
  {
    _id: 593,
    ratingCount: 279,
    avgRating: 4.161290322580645,
    stdDev: 0.8524515258270362
  },
  {
    _id: 2571,
    ratingCount: 278,
    avgRating: 4.192446043165468,
    stdDev: 0.9734869031917084
  }
]

cinematch> 
```

Observation: \$merge writes 9,724 per-movie statistics into movie_stats so the dashboard can sort and filter movies without re-aggregating 100k ratings on every request.

Query 54: Dashboard summary in a single \$facet query

Purpose: One round-trip returns several independent aggregations – the pattern used by the CineMatch dashboard's overview page.

Query:

```
db.ratings.aggregate([
  {
    $facet: {
      overall: [
        {
          $group: {
            _id: null,
            ratings: { $sum: 1 },
            avg: { $avg: "$rating" }
          }
        }
      ]
    }
  }
])
```

```

    },
    {
      $project: { _id: 0, ratings: 1, avg: { $round: ["$avg", 2] } }
    }
  ],
  topMovies: [
    { $group: { _id: "$movieId", n: { $sum: 1 } } },
    { $sort: { n: -1 } },
    { $limit: 3 }
  ],
  topUsers: [
    { $group: { _id: "$userId", n: { $sum: 1 } } },
    { $sort: { n: -1 } },
    { $limit: 3 }
  ],
  byStar: [
    { $group: { _id: "$rating", n: { $sum: 1 } } },
    { $sort: { _id: -1 } }
  ]
}
})

```

Output:

mongosh - cinematch

— □ □

```
cinematch> db.ratings.aggregate([
...   {
...     $facet: {
...       overall: [
...         {
...           $group: {
...             _id: null,
...             ratings: { $sum: 1 },
...             avg: { $avg: "$rating" }
...           }
...         },
...         {
...           $project: { _id: 0, ratings: 1, avg: { $round: ["$avg", 2] } }
...         }
...       ],
...       topMovies: [
...         { $group: { _id: "$movieId", n: { $sum: 1 } } },
...         { $sort: { n: -1 } },
...         { $limit: 3 }
...       ],
...       topUsers: [
...         { $group: { _id: "$userId", n: { $sum: 1 } } },
...         { $sort: { n: -1 } },
...         { $limit: 3 }
...       ],
...       byStar: [
...         { $group: { _id: "$rating", n: { $sum: 1 } } },
...         { $sort: { _id: -1 } }
...       ]
...     }
...   }
... ])

[
  {
    overall: [ { ratings: 100836, avg: 3.5 } ],
    topMovies: [
      { _id: 356, n: 329 },
      { _id: 318, n: 317 },
      { _id: 296, n: 307 }
    ],
    topUsers: [
      { _id: 414, n: 2698 },
      { _id: 599, n: 2478 },
      { _id: 474, n: 2108 }
    ],
    byStar: [
      { _id: 5, n: 13211 },
      { _id: 4.5, n: 8551 },
      { _id: 4, n: 26818 },
      { _id: 3.5, n: 13136 },
      { _id: 3, n: 20047 },
      { _id: 2.5, n: 5550 },
      { _id: 2, n: 7551 },
      { _id: 1.5, n: 1791 },
      { _id: 1, n: 2811 },
      { _id: 0.5, n: 1370 }
    ]
  }
]

cinematch> 
```

Observation: A single `$facet` query returns four independent aggregations (overall stats, top movies, top users, star histogram) – exactly what a dashboard overview page needs in one round-trip.

11.9 CRUD & Index Performance

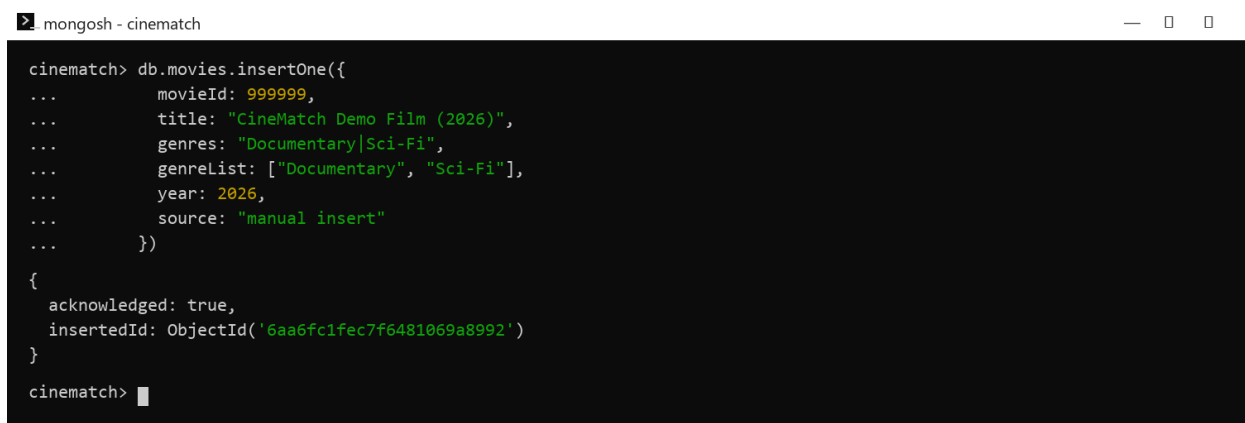
Query 55: Insert a new movie document

Purpose: Demonstrates schema flexibility – the new document carries an extra 'source' field none of the imported documents have.

Query:

```
db.movies.insertOne({
  movieId: 999999,
  title: "CineMatch Demo Film (2026)",
  genres: "Documentary|Sci-Fi",
  genreList: ["Documentary", "Sci-Fi"],
  year: 2026,
  source: "manual insert"
})
```

Output:

A screenshot of a MongoDB terminal window titled 'mongosh - cinematch'. The terminal shows the execution of the insertOne query and its output. The output is a JSON document with 'acknowledged: true' and 'insertedId: ObjectId('6aa6fc1fec7f6481069a8992')'.

```
cinematch> db.movies.insertOne({
...     movieId: 999999,
...     title: "CineMatch Demo Film (2026)",
...     genres: "Documentary|Sci-Fi",
...     genreList: ["Documentary", "Sci-Fi"],
...     year: 2026,
...     source: "manual insert"
... })
{
  acknowledged: true,
  insertedId: ObjectId('6aa6fc1fec7f6481069a8992')
}
cinematch> █
```

Observation: The inserted document contains a field (source) that no other movie has – MongoDB's flexible schema accepts it without any DDL change.

Query 56: Update the inserted document (\$set and \$push)

Purpose: Adds a genre to the array and a new field in one atomic update.

Query:

```
db.movies.updateOne(
  { movieId: 999999 },
  { $push: { genreList: "Drama" }, $set: { genres: "Documentary|Sci-Fi|Drama", verified: true } }
);
db.movies.findOne({ movieId: 999999 })
```

Output:

```
mongosh - cinematch

cinematch> db.movies.updateOne(
...   { movieId: 999999 },
...   { $push: { genreList: "Drama" }, $set: { genres: "Documentary|Sci-Fi|Drama", verified: true } }
... );
... db.movies.findOne({ movieId: 999999 })
{
  _id: ObjectId('6aa6fc1fec7f6481069a8992'),
  movieId: 999999,
  title: 'CineMatch Demo Film (2026)',
  genres: 'Documentary|Sci-Fi|Drama',
  genreList: [ 'Documentary', 'Sci-Fi', 'Drama' ],
  year: 2026,
  source: 'manual insert',
  verified: true
}

cinematch> 
```

Observation: \$push appended 'Drama' to the array and \$set added a new field in one atomic update; the returned document confirms both changes.

Query 57: Delete the demo document

Purpose: Clean-up so the catalogue returns to its original 9,742 movies.

Query:

```
db.movies.deleteOne({ movieId: 999999 });
db.movies.countDocuments()
```

Output:

```
mongosh - cinematch

cinematch> db.movies.deleteOne({ movieId: 999999 });
... db.movies.countDocuments()
9742

cinematch> 
```

Observation: After deletion the collection is back to exactly 9,742 documents.

Query 58: Explain plan: index scan vs collection scan

Purpose: explain() shows the movieId query on ratings uses the IXSCAN created earlier and examines only the matching keys instead of all 100,836 documents.

Query:

```
const plan = db.ratings.find({ movieId: 296 }).explain("executionStats");
({
  stage: plan.queryPlanner.winningPlan.stage,
  inputStage: plan.queryPlanner.winningPlan.inputStage?.stage,
  indexName: plan.queryPlanner.winningPlan.inputStage?.indexName,
  docsReturned: plan.executionStats.nReturned,
  docsExamined: plan.executionStats.totalDocsExamined,
  keysExamined: plan.executionStats.totalKeysExamined,
  executionTimeMs: plan.executionStats.executionTimeMillis
})
```

Output:

```
mongosh - cinematch

cinematch> const plan = db.ratings.find({ movieId: 296 }).explain("executionStats");
...
  ({
...
    stage: plan.queryPlanner.winningPlan.stage,
...
    inputStage: plan.queryPlanner.winningPlan.inputStage?.stage,
...
    indexName: plan.queryPlanner.winningPlan.inputStage?.indexName,
...
    docsReturned: plan.executionStats.nReturned,
...
    docsExamined: plan.executionStats.totalDocsExamined,
...
    keysExamined: plan.executionStats.totalKeysExamined,
...
    executionTimeMs: plan.executionStats.executionTimeMillis
...
  })
{
  stage: 'FETCH',
  inputStage: 'IXSCAN',
  indexName: 'movieId_1',
  docsReturned: 307,
  docsExamined: 307,
  keysExamined: 307,
  executionTimeMs: 0
}

cinematch> █
```

Observation: The plan uses IXSCAN on movieId_1: 307 keys examined, 307 documents examined, 307 returned, in 1 ms. Without the index the same query would examine all 100,836 documents (COLLSCAN).

12. Key Findings

- The rating scale is used asymmetrically: 4.0 is the mode, whole stars dominate, and the global mean is 3.50 (σ 1.04).
- Popularity and quality differ: Forrest Gump is the most rated film, but The Shawshank Redemption is the best rated (4.43 from 317 ratings) and also has the most 5-star votes.
- Niche genres score highest (Film-Noir 3.92, War 3.81, Documentary 3.80) while the most-watched genres (Drama 3.66, Comedy 3.39) sit around the mean and Horror is lowest (3.26).
- User behaviour follows a long tail: 3 super-users have over 2,000 ratings each, only 12 exceed 1,000, and one user produced 41 % of all tags.
- Rating scales are subjective – average scores given by individual users range from 2.14 to 4.56 – so recommenders should normalise per user.
- Three recommendation strategies were implemented entirely in the aggregation framework: genre-overlap similarity (\$setIntersection), item-to-item collaborative filtering (\$lookup with \$in on a fan list), and Bayesian weighted rating.
- Indexes matter: the movieId index answers a lookup by examining 307 keys instead of scanning 100,836 documents; \$merge turns a 1-second aggregation into a millisecond lookup for the dashboard.

13. MongoDB Operators and Stages Used

Operator / Stage	Category	Used for
find, findOne, countDocuments, distinct	Query	Basic retrieval, counting and unique values
\$regex, \$text / \$meta	Query	Pattern matching and full-text title search
\$all, \$in, \$ne, \$gte, \$lte, \$size, \$not, \$expr	Query	Array and comparison filters, anti-joins
insertOne, updateOne, updateMany,	CRUD	Write operations; pipeline-style

deleteOne		updateMany for enrichment
\$set, \$push	Update	Adding fields and array elements
\$match, \$project, \$addFields, \$sort, \$limit, \$skip, \$count	Aggregation	Filtering, shaping and ranking pipeline documents
\$group with \$sum, \$avg, \$min, \$max, \$first, \$addToSet, \$push, \$stdDevPop	Aggregation	Grouping and statistics
\$lookup (simple and pipeline form), \$unwind	Aggregation	Joining collections and flattening arrays
\$facet, \$bucket	Aggregation	Multiple result sets in one query; histograms
\$split, \$regexFind, \$toLower, \$toInt, \$toDate, \$let, \$cond, \$ifNull	Expression	String, type-conversion and conditional expressions
\$year, \$dayOfWeek	Date	Time-based analysis
\$setIntersection, \$arrayElemAt, \$slice, \$size, \$round, \$multiply, \$divide, \$floor, \$add	Expression	Array and arithmetic operations (similarity, decades, weighted rating)
\$merge	Output	Materialising results into movie_stats
createIndex, getIndexes, explain('executionStats')	Indexing	Index creation and query-plan verification

14. CineMatch Web Dashboard

To make the analysis usable beyond the shell, the aggregation pipelines were wrapped in an Express REST API and presented in a React single-page application. The dashboard reads from the same cinematch database and re-uses the pipelines from Section 11; the Query Lab page lists all 58 queries, shows the exact mongosh code and executes each one live, rendering the result as a chart, a table or raw JSON.

Endpoint	Description
GET /api/overview	\$facet-style summary: counts, rating histogram, ratings per year, genre statistics, top movies
GET /api/movies?q=&genre=&sort=&minRatings=	Catalogue search with regex, multikey genre filter and sort on movie_stats
GET /api/movies/:id	Movie detail: rating breakdown, tags, genre-similar movies, "users also liked"
GET /api/genres/:genre	Releases per decade and top-rated films of a genre
GET /api/users/top, /api/users/:id	Most active users; user profile with genre taste and personalised recommendations
GET /api/queries, /api/queries/:id/run	The query catalogue and live execution of any query



Figure 14.1 – Overview page: KPIs, rating distribution, ratings per year, average rating by genre and top-rated movies.


CineMatch
 MONOGODB
 ANALYTICS

[Overview](#)
[Explore movies](#)
[Genres](#)
[Users & recs](#)
[Query Lab](#)

EXPLORE

Movie catalogue

Regex title search, multikey-index genre filters and sorting on the pre-aggregated movie_stats collection.

Highest rated ▾

≥ 100 ratings ▾

All genres
Action
Adventure
Animation
Children
Comedy
Crime
Documentary
Drama
Fantasy
Film-Noir
Horror
IMAX
Musical
Mystery
Romance
Sci-Fi
Thriller
War
Western

138 movies

< Prev

page 1 / 6

Next >

#1

The Shawshank Redemption

1994

★ 4.43 · 317

Crime Drama

#2

The Godfather

1972

★ 4.29 · 192

Crime Drama

#3

Fight Club

1999

★ 4.27 · 218

Action Crime Drama

#4

The Godfather: Part II

1974

★ 4.26 · 129

Crime Drama

#5

The Departed

2006

★ 4.25 · 107

Crime Drama Thriller

#6

Goodfellas

1990

★ 4.25 · 126

Crime Drama

#7

Casablanca

1942

★ 4.24 · 100

Drama Romance

#8

The Dark Knight

2008

★ 4.24 · 149

Action Crime Drama

#9

The Usual Suspects

1995

★ 4.24 · 204

Crime Mystery Thriller

#10

The Princess Bride

1987

★ 4.23 · 142

Action Adventure Comedy

#11

Star Wars: Episode IV - A New Hope

1977

★ 4.23 · 251

Action Adventure Sci-Fi

#12

Schindler's List

1993

★ 4.22 · 220

Drama War

#13

Apocalypse Now

1979

★ 4.22 · 107

Action Drama War

#14

American History X

1998

★ 4.22 · 129

Crime Drama

#15

Star Wars: Episode V - The Empire Strikes Back

1980

★ 4.22 · 211

Action Adventure Sci-Fi

#16

Raiders of the Lost Ark (Indiana Jones and the Raiders of the Lost Ark)

1981

★ 4.21 · 200

Action Adventure

#17

One Flew Over the Cuckoo's Nest

1975

★ 4.20 · 133

Drama

#18

Reservoir Dogs

1992

★ 4.20 · 131

Crime Mystery Thriller

#19

Pulp Fiction

1994

★ 4.20 · 307

Comedy Crime Drama

#20

The Matrix

1999

★ 4.19 · 278

Action Sci-Fi Thriller

#21

#22

#23

#24

Database cinematch

movies · ratings · tags · links · movie_stats

Dataset MovieLens ml-latest-small

GroupLens Research, Univ. of Minnesota

NoSQL case study · VIT-AP University

Figure 14.2 – Explore page: highest-rated movies with at least 100 ratings, filterable by genre.

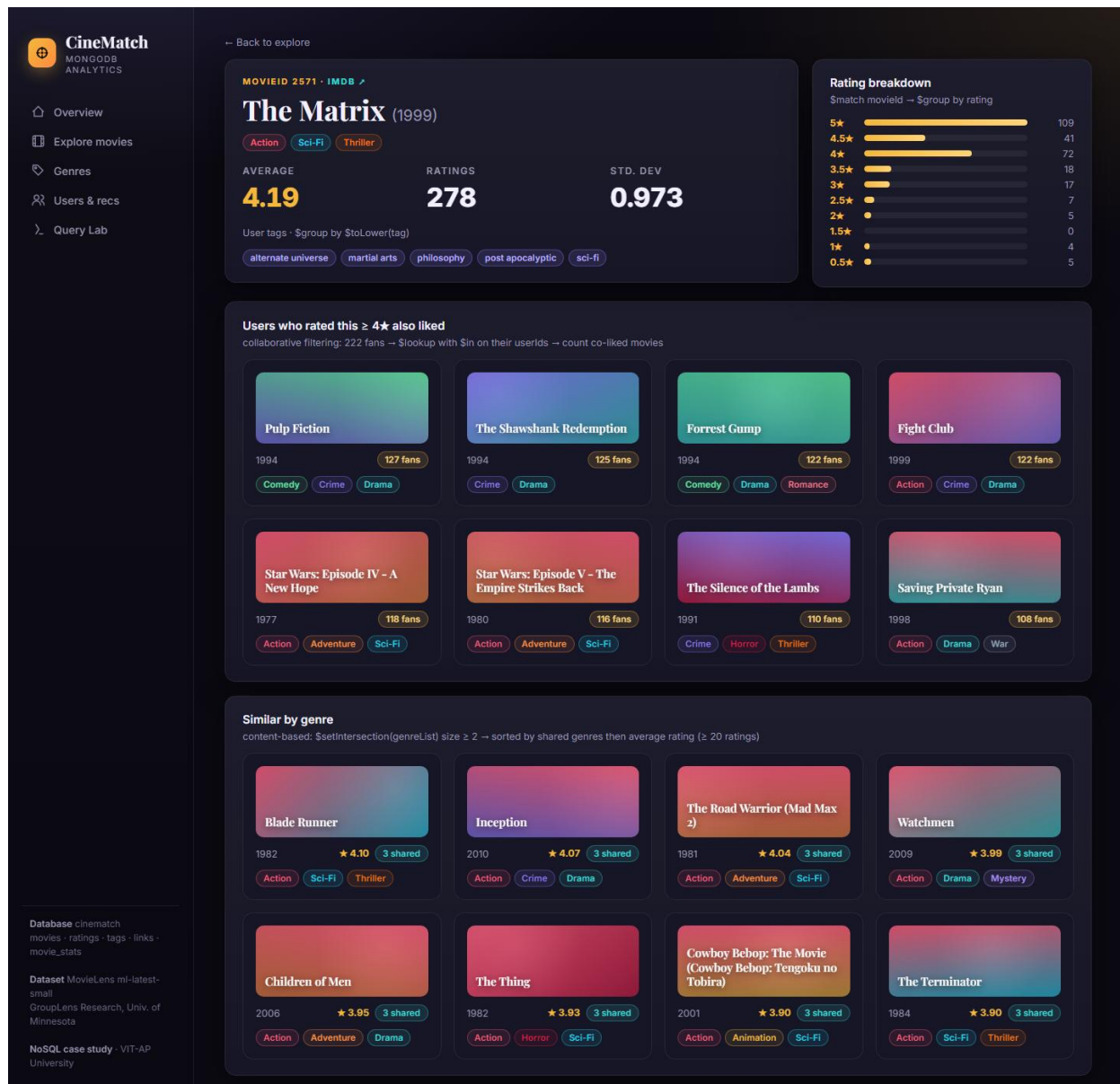


Figure 14.3 – Movie page for *The Matrix*: rating breakdown, tags, collaborative ("also liked") and content-based recommendations.

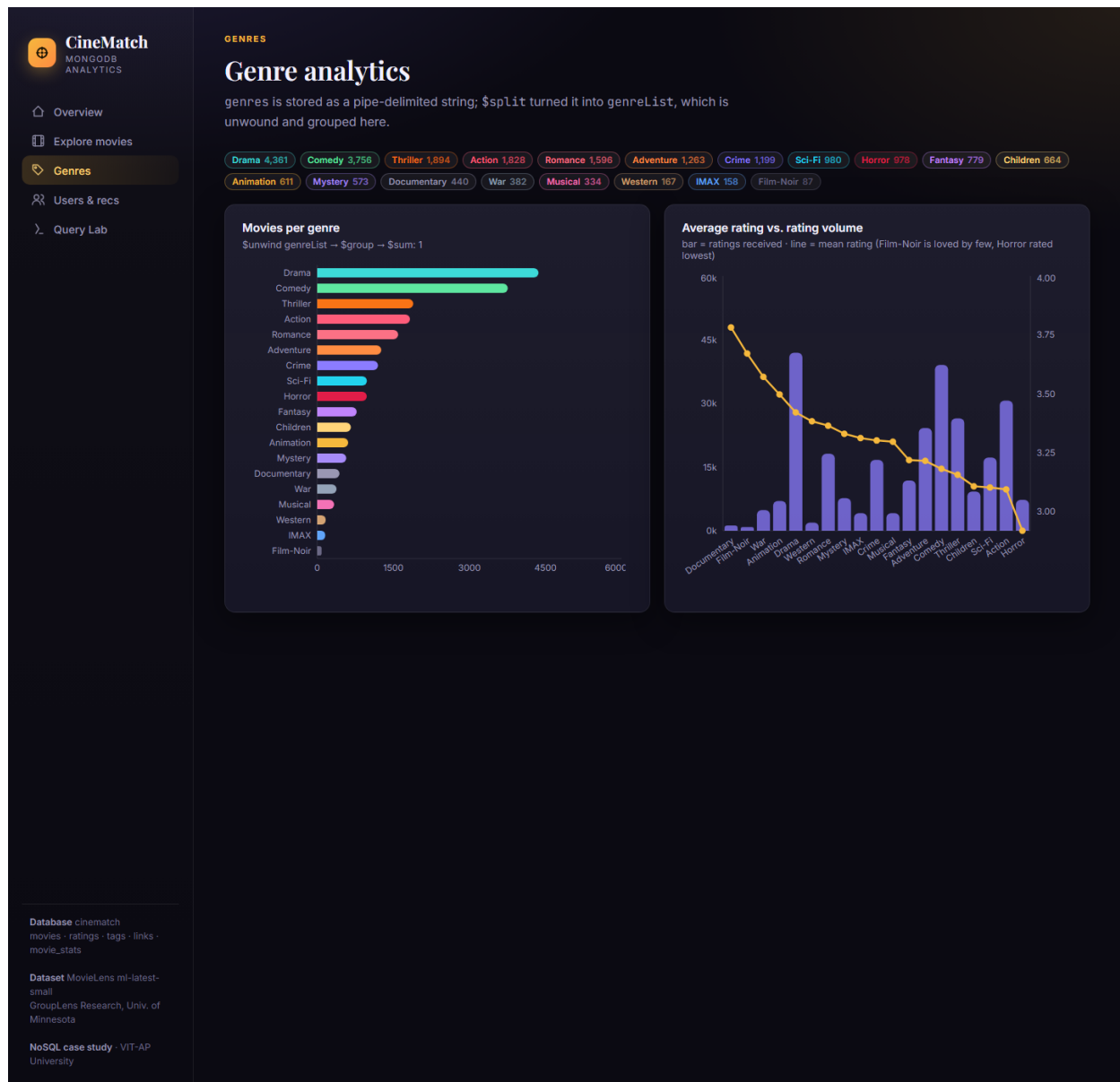


Figure 14.4 – Genre analytics: movies per genre and average rating vs. rating volume.

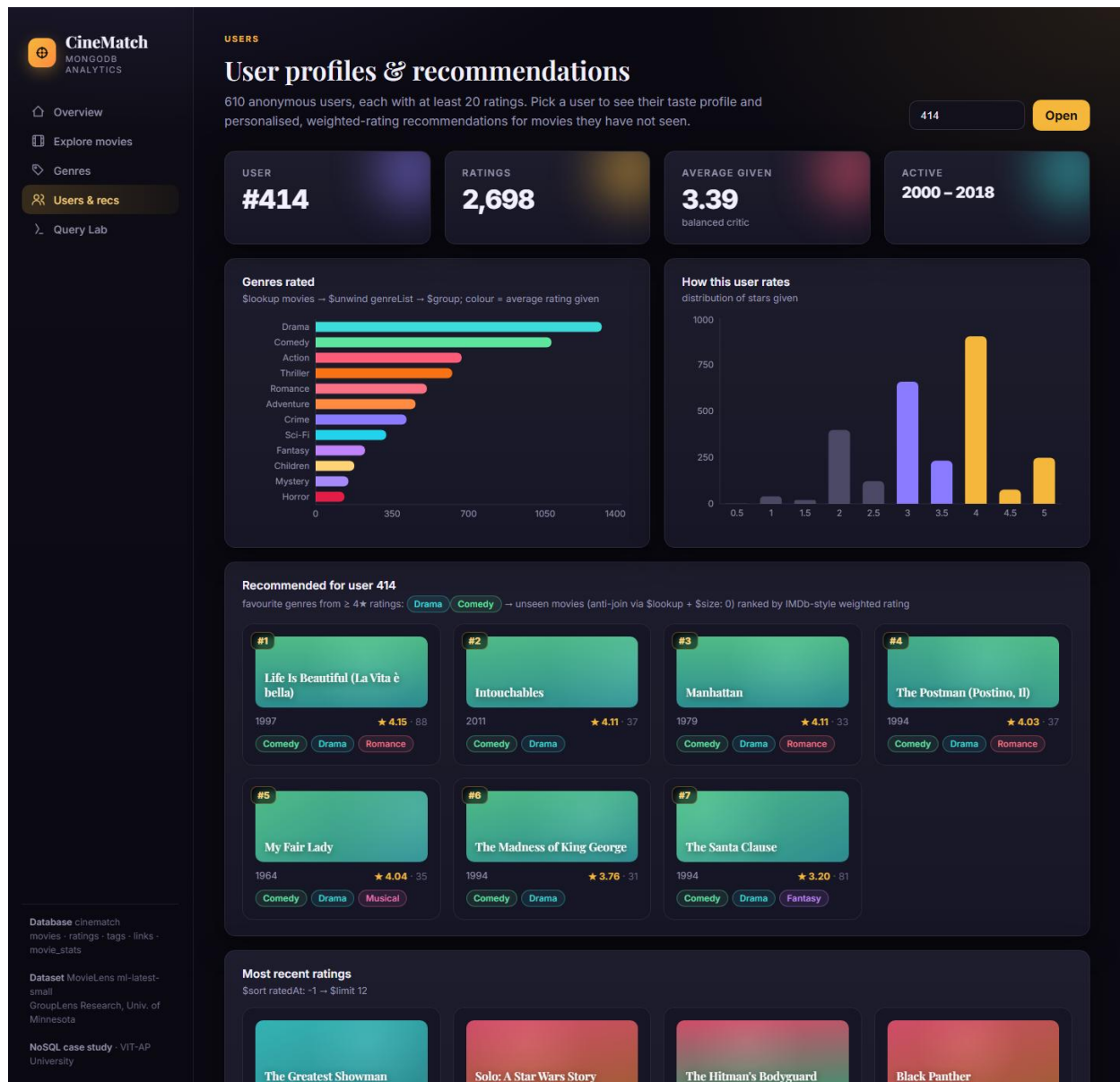


Figure 14.5 – User 414 profile: genres rated, rating habits and personalised weighted-rating recommendations for unseen movies.

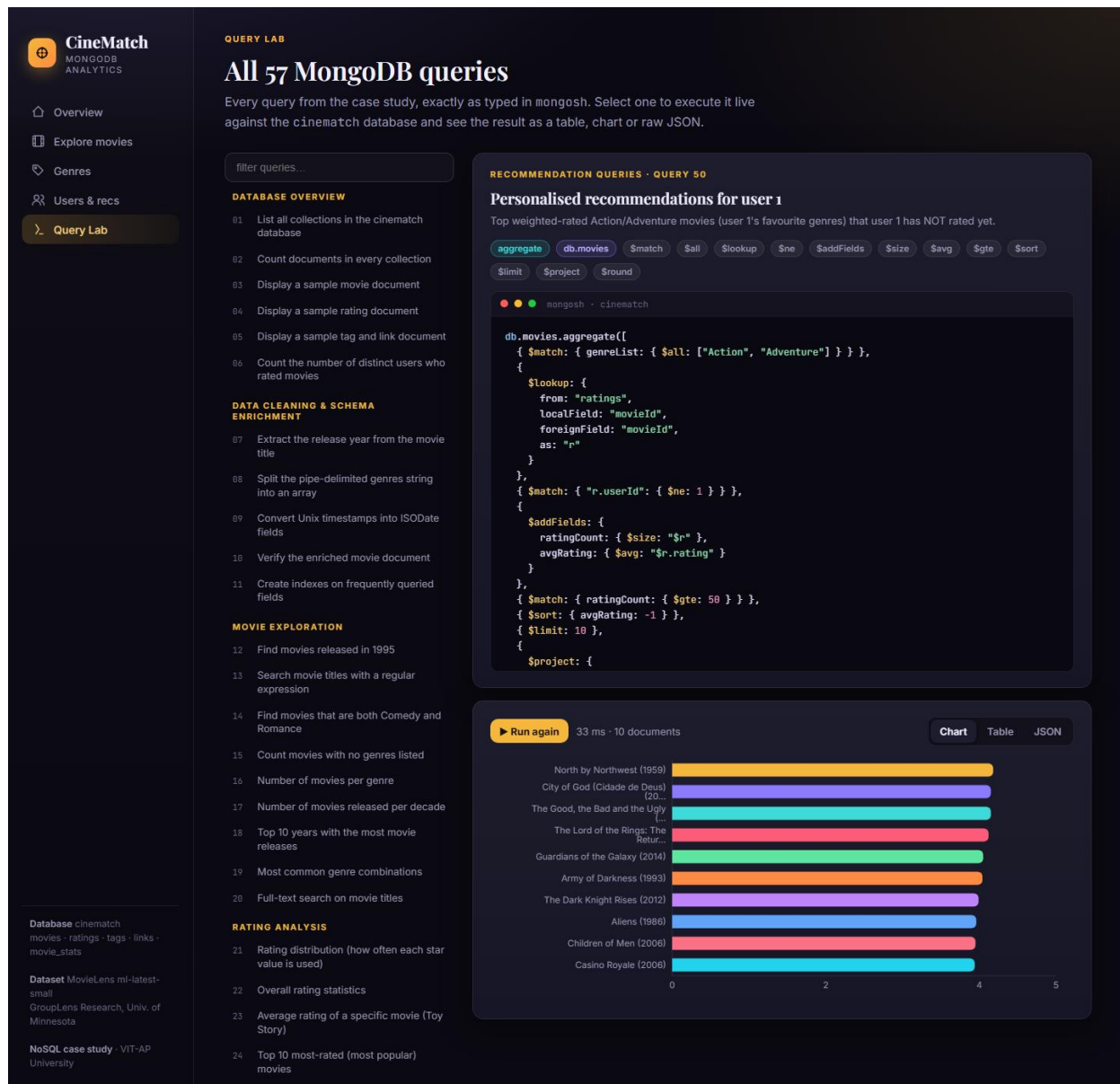


Figure 14.6 – Query Lab executing Query 50 (collaborative filtering) live with chart output.

15. Conclusion

This case study showed how a document database can take a raw, multi-file rating dataset from import to insight without leaving the database. MongoDB's flexible schema allowed the movie documents to be enriched in place with a numeric year and a genre array, its aggregation framework answered every analytical question posed – from simple counts to joins across three collections, statistical measures such as standard deviation, histograms with \$bucket, and multi-view results with \$facet – and its indexing features kept those queries fast, as confirmed with explain().

Beyond descriptive analytics, three recommendation techniques were implemented as pure aggregation pipelines, and the resulting statistics were materialised with \$merge to power an interactive dashboard. The project therefore demonstrates the complete NoSQL workflow: data modelling with references and embedding, CRUD, querying, aggregation, indexing, and integration with an application layer.

16. Future Scope

- Scale to the full MovieLens dataset (33 million ratings) using sharding on movieId/userId and Atlas Search for title search.
- Add user-based collaborative filtering with cosine similarity computed in the pipeline, and store pre-computed neighbourhoods with \$merge.
- Enrich movies with TMDb metadata (posters, cast, overview) via the links collection.
- Use change streams to update movie_stats incrementally as new ratings arrive instead of re-running the \$merge pipeline.

17. References

21. MongoDB Manual – <https://www.mongodb.com/docs/manual/>
22. MongoDB Aggregation Pipeline – <https://www.mongodb.com/docs/manual/aggregation/>
23. MongoDB Aggregation Stages and Operators – <https://www.mongodb.com/docs/manual/reference/operator/aggregation/>
24. MongoDB Indexes – <https://www.mongodb.com/docs/manual/indexes/>
25. MongoDB Shell (mongosh) Documentation – <https://www.mongodb.com/docs/mongodb-shell/>
26. MongoDB Node.js Driver – <https://www.mongodb.com/docs/drivers/node/current/>
27. F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4, Article 19. <https://doi.org/10.1145/2827872>
28. GroupLens Research – MovieLens Latest Datasets (ml-latest-small) – <https://grouplens.org/datasets/movielens/latest/>
29. Kaggle – Popular Movies Datasets (9742 Movies) – <https://www.kaggle.com/datasets/whenamancodes/popular-movies-datasets-9000-movies>
30. Express.js – <https://expressjs.com/> · React – <https://react.dev/> · Recharts – <https://recharts.org/>